

ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

**AGGRESSIVE FLIGHT TRAJECTORY GENERATION AND CONTROLLER
DESIGN FOR FLIP MANEUVER OF CRAZYFLIE2.1**

CONTROL AND AUTOMATION ENGINEERING DESIGN II

Muhammed Emin HAMAMCI

Control and Automation Engineering

Thesis Advisor: Doç. Dr. Ilker USTOGLU

JANUARY 2025

ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

**AGGRESSIVE FLIGHT TRAJECTORY GENERATION AND CONTROLLER
DESIGN FOR FLIP MANEUVER OF CRAZYFLIE2.1**

CONTROL AND AUTOMATION ENGINEERING DESIGN II

Muhammed Emin HAMAMCI
(040210748)

Kontrol ve Otomasyon Mühendisliği

Control and Automation Engineering

JANUARY 2025

Rahman ve Rahim Olan Allah'ın Adıyla,

*Yaptıkları fedakarlıklar, cesaretleri ve inançlarının yanında onlara destek olmak,
lkemize coğrafyamıza ve insanlığa faydalı bir mhendis ve ilim insanı olarak
kendimi yetiřtirmek iin gerekleřtirdiėim bu projeyi, bu toprakları vatan toėraėı
kılmak iin canlarını gzlerini kırpmadan veren btn řehit, gazilerimize ve
Gazze'de zulme karřı gės gerip mcadele eden kardeřlerime ithaf ediyorum ...*

PREFACE

İstanbul Teknik Üniversitesi Kontrol ve Otomasyon Mühendisliği bölümü lisans programında geçirdiğim yılların ardından edindiğim tecrübeler ve bilgi birikimim ile bu projeyi yapmak bana nasip oldu. Bir teknik üniversiteli olarak kendimi dersler ile sınırlı tutmayıp çeşitli projelerde bulunarak kendimi geliştirmeye gayret ettim. Bu yolda bana mühendislik bakış açısı ve matematiği sevdirmek açısından katkı sunan Kamil Karaçuha hocama, kontrol mühendisi olma kültürünü ve bakış açısını o güvenilir haliyle ve öğrencilere bilgilerini bakış açılarını aktarma sevgisiyle bana kazandıran Mehmet Turan Söylemez hocama, matematiksel olarak daha ileri sistemler ile ilgilenmemi ve pratik yapmamızı derslerde sağlayan Emre Dinçel hocaya, bitirme projemde bana destek olup danışmanlığımı yapan İlker Üstoğlu öğretmenime ve ismini sayamadığım bütün bölüm öğretim üyelerine ve görevlilerine şükranlarımı sunuyorum. Bununla birlikte okul dışı projelerde bulunduğum, proje yönetimi tecrübeleri elde ettiğim ve çeşitli ödüller kazandığımız İTÜ Gamma takımına ve 2021-2024 yılları arasında beraber çalıştığım yol arkadaşlarıma, bu serüvene atılmamda yardımcı olan 2021 yılı İTÜ Kasva takımı ve üyelerine teşekkürlerimi sunuyorum. Bu çalışmada istediğim hedeflere tam manasıyla ulaşamasam da sağlam adımlarla ve yaptığım işi doğru yaparak derinlemesine bir bilgi birikimi elde ederek gerçekleştirmeye çalıştım. İnşallah bu çalışma eğitim hayatımın devam etmesi durumunda üzerine çalışılabilecek bir örnek oluşturabilmiştir.

January 2025

Muhammed Emin Hamamcı

CONTENTS

Sayfa

PREFACE	vi
CONTENTS	viii
ABBREVIATIONS	x
SYMBOLS	xii
LIST OF TABLES	xiii
LIST OF FIGURES	xv
ÖZET	xviii
SUMMARY	xix
1. INTRODUCTION	1
1.1 Flip Maneuver	3
1.2 Main Objectives	4
1.3 Secondary Objectives	5
2. MATHEMATICAL MODEL OF CRAZYFLIE	7
2.1 Coordinate Frames	7
2.2 Kinematics	11
2.3 Dynamics	13
2.3.1 Force Equations - Translational Motion	13
2.3.2 Momentum Equations - Rotational Motion	14
2.4 Mathematical Model Parameters	17
2.4.1 Thrust Model	17
2.4.2 Torque	18
2.4.3 Result	19
2.5 State Space Model Representation	20
3. SIMULATION	22
3.1 Quadrotor Model	22
3.1.1 Propeller Inertia Calculation	26
3.1.2 Damping Coefficient of Rotor	26
3.1.3 Drag Coefficient of Quadrotor	27
3.2 6DOF Joint and State Estimation	27
4. CONTROLLER DESIGN	35
4.1 Outer Loop Position Controller	36
4.1.1 Altitude Controller	38
4.1.2 X-Y Controller	39
4.2 Inner Loop Controller	43
4.2.1 Attitude & Rate Controller	43
4.3 Motor Mixer Algorithm	46
5. SIMULATION RESULT	48
6. CONCLUSION	58
REFERENCES	60

ABBREVIATIONS

CAD	: Computer Aided Design
CCW	: Counterclockwise
CW	: Clockwise
DCM	: Direction Cosine Matrix
DOF	: Degree of Freedom
EKF	: Extended Kalman Filter
iLQR	: Iterative Lineae Quadratic Regulator
LQR	: Linear Quadratic Regulator
PID	: Proportional-Integral-Derivative Controller
PWM	: Pulse-Width Modulation
RPM	: Revolutions per minute
UAV	: Unmanned Aerial Vehicle

SYMBOLS

ϕ	: Vehicle Roll Angle [rad]
θ	: Vehicle Pitch Angle [rad]
Ψ	: Vehicle Yaw Angle [rad]
Ω	: Thrust Command [16-bit]

LIST OF TABLES

Sayfa

Table 1 : Coordinate frames transformations.	10
Table 2 : State variables for MAV equations of motion.	11
Table 3 : Model Parameters.	19
Table 4 : State variables for MAV equations of motion.	34
Table 5 : Altitude PID controller parameters.	39
Table 6 : Position X, P Controller.	42
Table 7 : Velocity X, PI Controller.	42
Table 8 : Position Y, P Controller.	43
Table 9 : Velocity Y, PI Controller.	43
Table 10 : Pitch and Roll Controller, PI Controller.	45
Table 11 : Pitch and Roll Rate Controller, PID Controller.	45
Table 12 : Yaw Controller, PID Controller.	45
Table 13 : Yaw Rate Controller, PI Controller.	46

LIST OF FIGURES

Sayfa

Figure 1 : CrazyFlie 2.1.	1
Figure 2 : Flip Maneuver.....	3
Figure 3 : Body-fixed frame and Inertial (World) frame.....	8
Figure 4 : Actual Body Frame.....	9
Figure 5 : “+” configuration at the left and “X” configuration at the right.....	10
Figure 6 : “X” configuration and motor number of CrazyFlie2.1.	15
Figure 7 : Simulink 6DOF joint and quadrotor model.....	22
Figure 8 : Quadrotor model subsystem block.	23
Figure 9 : CW direction motor.	23
Figure 10 : PWM To Thrust Force Function Block.....	24
Figure 11 : PWM To ω Function Block.	24
Figure 12 : CCW direction motor.....	24
Figure 13 : Rotor Simscape Implemented Model.	25
Figure 14 : Sample generated thrust, CCW direction.	25
Figure 15 : Rotor CAD model.	26
Figure 16 : Rotor CAD model.	26
Figure 17 : Resultant SimScape model.....	27
Figure 18 : 6DOF joint first section.....	28
Figure 19 : 6DOF joint translational joint.....	29
Figure 20 : 6DOF revolute joint.....	30
Figure 21 : 6DOF revolute joint.....	30
Figure 22 : “p” estimate.....	31
Figure 23 : “q” estimate.....	31
Figure 24 : “r” estimate.	32
Figure 25 : “u” estimate.....	32
Figure 26 : “v” estimate.....	33
Figure 27 : “w” estimate.....	33
Figure 28 : Cascaded PID controller.....	35
Figure 29 : Block Diagram of CrazyFlie 2.1 Firmware Controller Scheme.....	36
Figure 30 : Outer loop position controller structure.	37
Figure 31 : Altitude PID controller.	38
Figure 32 : X-Y position controller.	39
Figure 33 : Position body error logic.....	40
Figure 34 : Position body error logic.....	41
Figure 35 : Position body error logic.....	42
Figure 36 : Attitude controller scheme.....	44
Figure 37 : Rate controller scheme.....	44
Figure 38 : Rate controller scheme.....	47
Figure 39 : Z position tracking for step input 1m.	48
Figure 40 : Z position error for step input 1m.	49
Figure 41 : For Z tracking step input 1m, control signal.	49
Figure 42 : Z position tracking with white noise added state estimation (0.001 power) step input 1m.	50
Figure 43 : For Z tracking with white noise added state estimation (0.001 power) step input 1m, control signal.....	50
Figure 44 : Reference Signal for position step tracking.	51

Figure 45 : X,Y,Z step tracking simulation.	51
Figure 46 : X,Y,Z step tracking performance.	52
Figure 47 : X,Y,Z step tracking error.	53
Figure 48 : X,Y,Z step tracking result body velocity.	53
Figure 49 : X,Y,Z step tracking control signals.....	54
Figure 50 : Helix trajectory tracking simulation.....	55
Figure 51 : Helix trajectory tracking performance.....	55
Figure 52 : Helix trajectory tracking error.	56
Figure 53 : Helix trajectory tracking result body velocity.	56
Figure 54 : Helix trajectory tracking control signals.....	57

TÜRKÇE TEZ BAŞLIĞI BURAYA YAZILIR

ÖZET

Bu bitirme projesi kapsamında bir Kontrol ve Otomasyon Mühendisliği öğrencisi olarak belirlenen bir proje için; dinamik sistem modelleme, sistem analizi, kontrolcü tasarımı, simülasyon ve test ortamlarının kurulumu ile gerçek zamanlı uygulama gibi işlemleri gerçekleştirme bilgisine sahip olduğu ortaya konacaktır. "CrazyFlie2.1 için Flip Manevrası Amacıyla Agresif Uçuş Yörüngesi Üretimi ve Kontrolcü Tasarımı" başlığı altında sıfırdan başlayarak kapsamlı bir çalışma gerçekleştirilecektir. Bu başlık kapsamında seçilen platformun tanıtılması, matematiksel modelleme, simülasyon kurulumu, hareket planlama ve çeşitli manevralar için kontrolcü tasarımı gibi çalışmalar yapılacaktır.

Bu çalışmanın birincil amacı, bir nano-quadcopter'in sistem modellemesini araştırmak ve pozisyon ile flip manevrası kontrol algoritmalarını tasarlamak olup, nihai hedef bu sistemin hem simülasyon hem de gerçek platformda test edilmesidir. Bu çalışmayı gerçekleştirmek için, açık kaynaklara sahip olan ve üzerinde çeşitli akademik çalışmalar yapılmış olan CrazyFlie2.1 platformu seçilmiştir. Seçilen platform için çeşitli kontrolcülerin tasarlanabilmesi ve Matlab Simulink aracılığıyla bir simülasyon kurulumu yapılabilmesi adına sistem modelleme çalışmaları gerçekleştirilmelidir. Sistem modellemesinin ardından, Simulink üzerindeki SimScape kütüphanesi kullanılarak bir simülasyon ortamı hazırlanacaktır. Gerekli temel ihtiyaçlar karşılandıktan sonra, nano-quadcopter'in gerçekleştirmesi istenen manevra veya hareket türlerinin belirlenmesi ve sınıflandırılması gerekmektedir. Sınıflandırmanın ardından, bu hareketlerin gerçekleştirilmesi için gerekli literatür taramalarının yapılması, her manevra için uygulanabilir tekniklerin belirlenmesi ve tüm bu tekniklerin aynı platformda gerçek zamanlı olarak çalıştırılıp çalıştırılamayacağının değerlendirilmesi gereklidir.

Kontrolcü tasarım süreçlerinde, öncelikle her bir manevra için gereksinimlerin belirlenmesi ve ardından çeşitli kontrolcülerin performanslarının simülasyon ortamında karşılaştırılması gereklidir. Temel ihtiyaçlar belirlendikten sonra, kontrolcü tasarımları teorik olarak yapılmalı ve sırasıyla simülasyon ve gerçek zamanlı testlere tabi tutulmalıdır. Bu özet sürecin tamamıyla, dinamik bir hava aracı için adım adım otonomi kazanım süreçleri ve farklı bakış açılarından teknikler değerlendirilecektir.

Sonuç olarak, bu çalışmanın amacı, CrazyFlie2.1 nano-quadcopter için modüler ve genişletilebilir bir otopilot sistemi ortaya koymaktır. Bu otopilot sistemi, temel kalkış, iniş, havada sabit durma, noktalar arası navigasyon ve nihayetinde agresif manevralar kapsamında Flip hareketini içerecektir. Tüm modelleme, tasarım ve simülasyon süreçlerinin ardından, bu otopilot sisteminin gereksinimleri yerine getirdiğini gerçek zamanlı olarak gözlemlemek hedeflenmektedir.

AGGRESSIVE FLIGHT TRAJECTORY GENERATION AND CONTROLLER DESIGN FOR FLIP MANEUVER OF CRAZYFLIE2.1

SUMMARY

For a project determined as a Control and Automation Engineering student within the scope of this graduation project study; It will be demonstrated on the subject in question that one has the knowledge to perform operations such as dynamic system modelling, system analysis, controller design, installation of simulation and test environments and real-time implementation. A comprehensive study will be carried out starting from scratch under the selected title "Aggressive Flight Trajectory Generation and Controller Design for Flip Maneuver of CrazyFlie2.1". Within the scope of this title, studies such as introducing the selected platform, mathematical modeling, simulation setup, motion planning and designing controllers for various maneuvers will be carried out.

The primary purpose of this study is to investigate the system modeling of a nanoquadcopter as well as designing position and flip maneuver control algorithms, with the ultimate goal of testing the system both in simulation and on a real platform. In order to carry out this study, the CrazyFlie2.1 platform, which has open sources and on which various academic studies have been conducted, was chosen. System modeling studies need to be carried out to design various controllers for the selected platform and establish simulation via Matlab Simulink. After the system modeling, a simulation environment will be prepared using the SimScape library on Simulink. After the necessary basic needs are met, the types of maneuvers/motions that the nano-quadcopter is desired to perform must be determined and classified. As a result of the classification, it is necessary to conduct the necessary literature searches to perform these movements, then determine the possible techniques for each maneuver and determine the necessary methods by considering whether all these techniques can be run in real time on the same platform. In the controller design processes, first the requirements for each maneuver must be determined, and then the performance comparison of various different controllers should be made in the simulation environment. After the basic needs are determined, the controller designs should be made theoretically and subjected to simulation and real-time tests, respectively. With this entire summary process, possible autonomy acquisition processes for a step-by-step dynamic aircraft and techniques from different perspectives will be evaluated.

As a result, the aim of this study is to reveal a modular and expandable autopilot system designed for the CrazyFlie2.1 nano-quadcopter. This autopilot system will include basic take-off, landing, hovering, inter-point navigation and finally, as the ultimate goal, Flip movement within the scope of aggressive maneuvering. After all modeling, design and simulation processes, it is aimed to observe in real time that this autopilot system fulfills the requirements.

1. INTRODUCTION

Drones and robots are gaining more and more importance in everyday life. They are used for search and rescue mission's photography from viewpoints that are difficult to reach for humans and countless other applications. Many of them already fly autonomously, however there are still limits making human intervention necessary on occasion. Indoors, autonomous operation in particular is possible etc. Moreover, for a variety of different tasks; Examples include drone races, military missions, and aggressive flights are also required.

Most of the drones that have been made so far traditionally consist of a small body and 4 engines (X or + configuration) and propellers placed at the same intervals around the body. For this reason, it is an underactuated system since it has 6DOF and 4 actuators. They do not weigh much. The aim is to move the body simply with 4 rotating engines. If we need to classify them, they can be divided into two: hardware/software.

CrazyFlie drones are a type of unmanned aerial vehicle (UAV) that are designed for use in research, education, and hobby applications. There are studies such as CrazySwarm [1] in the field of swarms especially focused on this drone. Implementation of swarm technology is also an area that contains various control problems. These UAVs are characterized by their small size, lightweight construction, and open-source design, which allows for flexibility in their programming and use.



Figure 1 : CrazyFlie 2.1.

When investigating whether this quadcopter has the flight dynamics to perform the Flip maneuver due to its small size, we can see a Flip test for CrazyFlie1.0 in the video in [2].

However, multiple studies on CrazyFlie2.1 can be found in the literature. To mention a few of the important ones, the first one is a doctoral thesis covering system identification processes on CrazyFlie2.0 [3]. Thanks to the studies carried out in this thesis, it was possible to develop a new model for the Extended Kalman Filter (EKF) included in the CrazyFlie 2.0 firmware. However, we can say that this article created the CrazyFlie2.0 firmware using this official study and formed the firmware basis for CrazyFlie2.1. Another important study is the study that includes end-to-end modeling and controller design processes for CrazyFlie2.0 [4]. In addition to these two main studies that attract attention, many more studies can be accessed.

Classical controllers (e.g., PID, LQR, iLQR) in conjunction with analytic models for the rigid body dynamics of a quadrotor are often sufficient to control vehicle attitude. In addition, linearized models are sufficient to simultaneously control for global trajectory attitude setpoints using well-tuned nested PID controllers. Standard control approaches may be show impressive acrobatic performance with quadrotors if well tuned. As long as non-linear situations are not involved too much (extreme case etc.), linear models and controllers can perform well. However, as aggressive flight concepts become more involved, the aim is to move away from linearity and make the most efficient flight. In other words, this actually requires taking advantage of all the dynamics of the vehicle.

It represents an area that includes both control and optimization within the aggressive flight problem and today Artificial Intelligence applications. Various orbiting techniques can be used for aggressive flight. However, various controllers can be designed to follow these created trajectories.

In classical orbit generation algorithms, trajectories can be generated and controlled by solving optimization problems using simplified vehicle dynamic models. However, as the aggressiveness increases, the complexity of the problem increases and tools such as classical PID design and Feedback Control Theory begin to become insufficient.

A plan must be made using all the dynamics of the existing vehicle. In classical methods, as aggressiveness increases, the margin of error also increases, and new techniques need to be developed for this. It can be concluded that several effective methods exist for trajectory generation of isolated maneuvers (backflip, flip, point-to-point) and performance measures (minimum time). This has been achieved primarily by using simplified planar models. Nowadays, solutions to these problems are sought with Reinforcement Learning, Optimal Control, Non-Linear Analysis, Intelligent Control and Adaptive Control techniques [5-10].

1.1 Flip Maneuver

There are various studies on the flip maneuver in the literature. There are various studies in the literature on the CrazyFlie2.0 or 1.0 drone, and the studies differ from each other. As shown in Figure 2, a multi-flip maneuver is a process that can be divided into three phases, climb, multi-flip, and descent and re-stabilization.

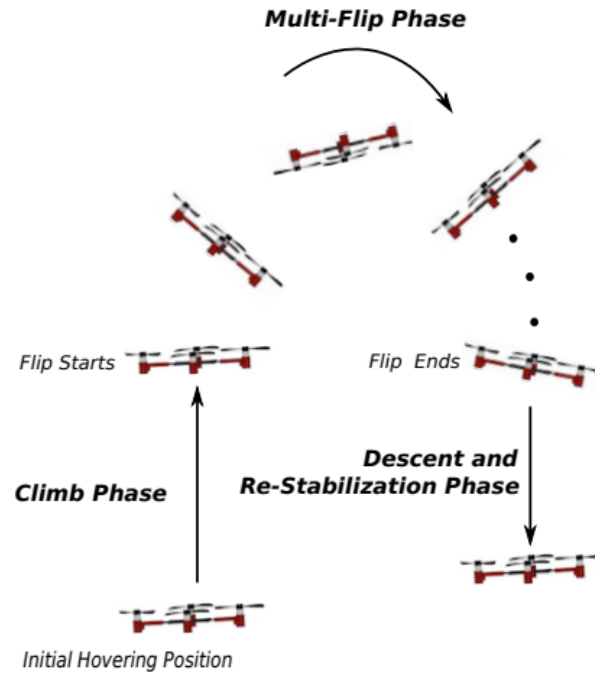


Figure 2 : Flip Maneuver.

At the beginning, the quadrotor is hovering at a certain altitude and then is triggered to start the multi-flip maneuver. More formal definition of Flip Maneuver given in [11]. The type of maneuver is explained by dividing it into stages.

- **Acceleration:** Accelerate up at near-maximum collective acceleration while rotating slightly away.
- **Start Rotate:** Use maximum differential thrust to achieve $\dot{\theta}_{max}$.
- **Coast:** Hold $\dot{\theta}_{max}$ (use a low collective thrust command to prevent accelerating into ground).
- **Stop Rotate:** Maximum differential thrust to reach $\dot{\theta}$ slightly less than 0.
- **Recovery:** Accelerate up with near-maximum collective thrust with a slight rotational acceleration to stop vertical descent and any remaining horizontal movement.

In this defined maneuver, it is assumed that the aircraft starts the maneuver from the hover position. However, in order to define a more general state-of-the-art method, the maneuver must be performed in any situation where the aircraft can perform this maneuver, that is, in any situation where a feasible solution can be found. As a result, solutions can be added in future studies that cover those different from the initial condition hovering position.

As a result, the Flip Maneuver actually constitutes a control problem in which the dynamics of the vehicle are challenged, unlike classical situations that involve more than one stage

1.2 Main Objectives

The main objectives of the research project were:

- Derivation of the general quadcopter mathematical model, derivation of kinematic and dynamic equations, definition of force-torque relationships.
- System identification using references to derive the CrazyFlie 2.1 model as a special subset of the general quadcopter model.
- Setting up the simulation environment via Matlab, Simulink.
- Performing basic quadcopter maneuvers (hover, position tracking, velocity tracking, attitude and attitude rate tracking etc.) using classical cascaded controller structures.

- Determining the requirements for performing aggressive maneuvers and evaluating the methods used in the literature.

1.3 Secondary Objectives

A number of smaller milestones were set to help the project achieve its key objectives:

- Literature summary.
- Examine and describe the control architecture within the Crazyflie's firmware.
- Making improvements for a better simulation. (Singularity condition check etc. Quaternions can be used)
- Simulation environment verification for aggressive maneuver tests.

2. MATHEMATICAL MODEL OF CRAZYFLIE

Making the mathematical model of CrazyFlie drone and even establishing the necessary infrastructure for simulation will facilitate the controller design studies. The task at hand involves developing mathematical models of the dynamics of CrazyFlie drones. This will require applying principles of control systems engineering to understand the behavior of these UAVs, including their motion and the interactions between the drones and their environment. The goal of this modeling exercise is to enable the design and implementation of control systems that can effectively stabilize and control the motion of CrazyFlie drones within the scope of position tracking, velocity tracking, attitude and attitude rate tracking. After these basic controllers are developed, structures such as different angular velocity and acceleration controllers can be added. In order to do all of this, it is necessary to understand the behavior of the system well. For this purpose, it makes sense to examine the complete mathematical model of the drone by dividing it into several parts and to run them all together. In this study, drone kinematic model, dynamical model, motor models and non-linear equation of motions can be derive. The Mathematical Model section depends on 2 major article and all its referenced sources [3,4].

In addition, it is possible to make the following assumptions that will not affect the result in the derivation of kinematic and dynamic equations:

- The quadcopter is a rigid body that cannot be deformed. It is possible to use general rigid-body dynamic equation.
- The quadcopter is symmetrical in its geometry.
- The mass is constant. Thus derivative of mass equals to zero.

2.1 Coordinate Frames

In this section, in order to define the equations of motion, reference axes must be defined according to Newton's principles. After years of developing studies on aviation, a common notation is generally used in coordinates frames definitions and equations of motion.

Let's start with the definition of the coordinate planes to be referenced first. We can talk about the necessity of 3 coordinate frames in order to define the necessary mathematical operations in three-dimensional space. These planes, in turn, can be defined as:

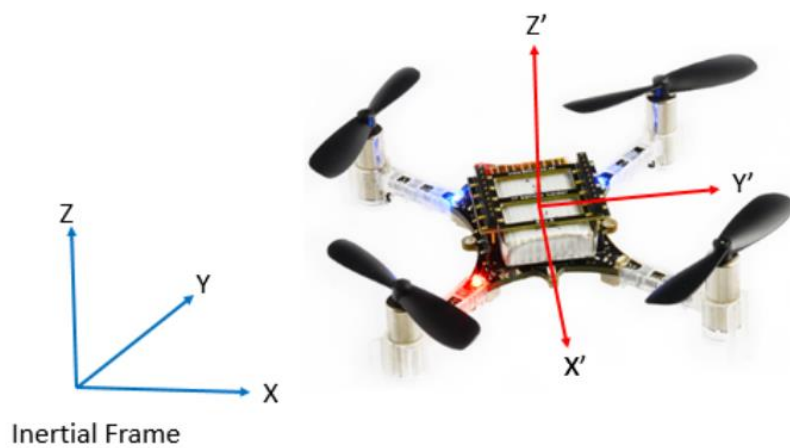


Figure 3 : Body-fixed frame and Inertial (World) frame.

- **World (Inertial) Frame:** It is the main plane to be referenced, and everything else to be defined has a counterpart in this coordinate plane and is fixed. Another name is Inertial Frame. Defined in three axes as X-Y-Z.
- **Fixed-Body (Vehicle) Frame:** According to the World Frame, it is the reference frame that corresponds to the drone's center of gravity. This coordinate plane, whose origin coincides with the center of gravity, does not rotate around any axis with the movements of the drone, that is, it always lies parallel to the World Frame. Where is the center of gravity of the drone and the speed of the drone relative to its axis, etc. definitions.
- **Reference Actual (Body) Frame:** The Actual Body Frame is used to describe the rotation movements of the drone. Its origin is again at the center of gravity. However, its difference from Fixed-Body (Vehicle) Frame defines the axis rotated by the roll, pitch, yaw angles around the fixed body frame of the drone. It is not parallel to the World Frame and defines the real state. In addition, another feature is that the motors produce thrust in the direction of the z-axis of this coordinate plane. In other words, according to the World Frame, the thrust is mostly not perpendicular to the World Frame during movement.

In aeronautic systems, there are two common conventions for defining the axes of a frame of reference: NED (North, East, Down) and ENU (East, North, Up). In NED frames, the Y axis points towards the east and the X axis points towards the true north, and a positive altitude is defined as downwards. In ENU frames, the Y axis points towards the north and the X axis points towards the east, and a positive altitude is defined as upwards. The Crazyflie 2.1 firmware follows the ENU convention, meaning that the Y axis points towards the north and the X axis points towards the east, and a positive altitude is defined as upwards.

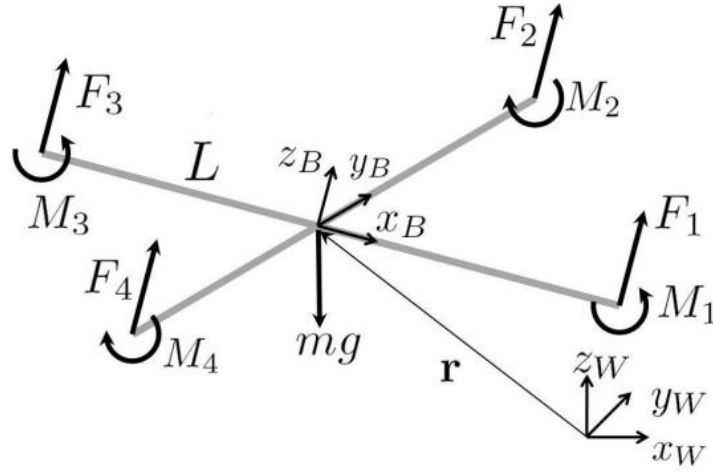


Figure 4 : Actual Body Frame.

Transition between these coordinate planes is accomplished by means of rotation matrices. In other words, any vector can be defined on all three planes after mathematical operations.

These mathematical operations are performed by means of rotation matrices. Another name used in applications is Direction Cosine Matrix (DCM). The DCM, representing the attitude of the body frame relative to the reference frame, is specified by a 3×3 rotation matrix R , the columns of which represent unit vectors in the body axes projected along the reference axes. Here, R_v^b is the rotation matrix operate transform like position vector, velocity vector, from intertial frame to body frame. R_v^b is written here in component form as:

$$\begin{bmatrix} \cos\theta \cos\psi & \cos\theta \sin\psi & -\sin\theta \\ \sin\phi \sin\theta \cos\psi - \cos\phi \sin\psi & \sin\phi \sin\theta \sin\psi + \cos\phi \cos\psi & \sin\phi \cos\theta \\ \cos\phi \sin\theta \cos\psi + \sin\phi \sin\psi & \cos\phi \sin\theta \sin\psi - \sin\phi \cos\psi & \cos\phi \cos\theta \end{bmatrix} \quad (2.1)$$

The rotation matrix given in Equation (2.1) is a combination of three different rotations. All operations between frames are given in Table 1.

Table 1 : Coordinate frames transformations.

Operation	From	Variable	Result
Trans – 1	Inertial Frame	$p_{cg/o} = [x, y, z]$	Vehicle Frame
Rot – 1	Vehicle Frame	ψ around k^v	Vehicle – 1
Rot – 2	Vehicle – 1	θ around j^{v1}	Vehicle – 2
Rot – 3	Vehicle – 2	ϕ around j^{v2}	Body Frame

Vehicle frame is a coordinate system whose x, y, z axes coincide with the inertial frame x, y, z axes formed by translating the inertial frame center to the aircraft's center of gravity. First a rotation of an angle ψ around the z axis of vehicle frame. Resultant frame called as Vehicle – 1 frame. Then a rotation of an angle θ around the Vehicle – 1 frame y axis and the resultant frame called as Vehicle – 2 . And finally a rotation of an angle ϕ around the Vehicle – 2 frame x axis. Where ϕ , θ and ψ represent the roll, pitch and yaw angles of the quadcopter's body. These rotation operations are defined as Euler Z-Y-X angles. All these axes mentioned are in the NED convention.

There are two configurations for the flight of a quadcopter: "configuration +" and "configuration X". These configurations refer to the orientation of the X-Y frame relative to the arms of the quadcopter. The orientation of the X-Y frame in "configuration +" is different from the orientation of the X-Y frame in "configuration X," as shown in Figure 5.

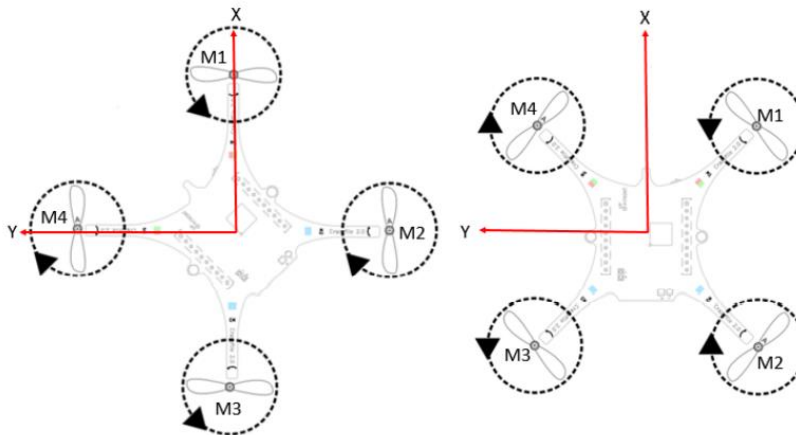


Figure 5 : “+” configuration at the left and “X” configuration at the right.

Also CrazyFlie 2.1 in the X frame structure. After these definitions, dynamic equations can be started to be analyzed.

2.2 Kinematics

In this section, defining the relations between positions and velocities of UAV will be described. In developing the equations of motion for a MAV, twelve state variables will be introduced. There are three position states and three velocity states associated with the translational motion of the MAV. Similarly, there are three angular position and three angular velocity states associated with the rotational motion. The state variables are listed in Table 2.

Table 2 : State variables for MAV equations of motion.

State	Description
x	X position of CoG in the inertial frame
y	Y position of CoG in the inertial frame
z	Z position of CoG in the inertial frame
ϕ	Roll angle
θ	Pitch Angle
ψ	Yaw Angle
u	Body frame velocity measured along i^b in F^b
v	Body frame velocity measured along j^b in F^b
w	Body frame velocity measured along k^b in F^b
p	Angular velocity measured along i^b in F^b
q	Angular velocity measured along j^b in F^b
r	Angular velocity measured along k^b in F^b

$$p_{CG/i} = [x, y, z]^T \quad (2.2)$$

$$V_{CG/i}^b = [u, v, w]^T \quad (2.3)$$

$$\omega_{b/i}^b = [p, q, r]^T \quad (2.4)$$

The translational velocity of the MAV is commonly expressed in terms of the velocity components along each of the axes in a body-fixed coordinate frame. The components of $V_{CG/i}^b$ u, v and w correspond to the inertial velocity of the vehicle projected onto the i^b, j^b, k^b axes, respectively. On the other hand, the translational position of the MAV is usually measured and expressed in an inertial reference frame. Relating the

translational velocity and position requires differentiation and a rotational transformation:

$$\frac{d}{dt} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = R_b^v \begin{pmatrix} u \\ v \\ w \end{pmatrix} = (R_v^b)^T \begin{pmatrix} u \\ v \\ w \end{pmatrix} \quad (2.5)$$

This is a kinematic relationship as it relates the derivative of position to velocity. Forces or accelerations are not considered [12].

The relationship between angular positions ϕ, θ, ψ and the angular rates p, q, r is also complicated by the fact that these quantities are defined in different coordinate frames. The angular rates are defined in the body frame F^b . The angular positions (Euler angles) are defined in three different coordinate frames:

- The roll angle ϕ is a rotation from F^{v2} to F^b about the $i^{v2} = i^b$.
- The pitch angle θ is a rotation from F^{v1} to F^{v2} about the $j^{v1} = j^{v2}$.
- The pitch angle ψ is a rotation from F^v to F^{v1} about the $k^v = k^{v1}$.

The body-frame angular velocities p, q, r can be expressed in terms of the derivatives of the Euler angles ϕ, θ, ψ , provided that the proper rotational transformations are carried out as follows:

$$\begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} \dot{\phi} \\ 0 \\ 0 \end{pmatrix} + R_{v2}^b(\phi) \begin{pmatrix} 0 \\ \dot{\theta} \\ 0 \end{pmatrix} + R_{v2}^b(\phi) R_{v1}^{v2}(\theta) \begin{pmatrix} 0 \\ 0 \\ \dot{\psi} \end{pmatrix} \quad (2.6)$$

Then the result become as:

$$\begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \sin\phi \cos\theta \\ 0 & -\sin\phi & \cos\phi \cos\theta \end{bmatrix} \begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix} \quad (2.7)$$

$$\begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix} = \begin{bmatrix} 1 & \sin\phi \tan\theta & \cos\phi \tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi \sec\theta & \cos\phi \sec\theta \end{bmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix}, \text{ for } \theta \neq \frac{\pi}{2} \quad (2.8)$$

In this section, the relationship between the position and velocity values is defined without considering the force and acceleration equations of the aircraft. First, the translational motion is defined via the kinematic equation (2.5). Then, the results of

the relationships between the angular positions and velocities are given in (2.7) and (2.8). Due to its clear and simple explanation, most of this section was written using [12].

2.3 Dynamics

To derive the dynamic equations of motion for the MAV, we will apply Newton's second law-first to the translational degrees of freedom and then to the rotational degrees of freedom. Newton's laws hold in inertial reference frames, meaning the motion of the body of interest must be referenced to a fixed (i.e., world, inertial) frame of reference, which in our case is the ground [12].

2.3.1 Force Equations - Translational Motion

Newton's second laws says that $f = ma$:

$$\Sigma f = m \frac{d}{dt_i} V_{CG/i}^b \quad (2.9)$$

Derivative of velocity can be rewrite using coriolis equation. In this equation m is mass and known velocity is measured on body. Then the coriolis equation expressed in body frame:

$$\Sigma f^b = m \left(\frac{d}{dt_b} V_{CG/i}^b + \omega_{b/i}^b \times V_{CG/i}^b \right) \quad (2.10)$$

The $f^b \triangleq [f_x, f_y, f_z]$ is the collection of all forces such as aerodynamic and thrust acting on the aircraft. Which they are defined in the body frame. After some computation, rearrange (2.10):

$$\begin{pmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{pmatrix} = \begin{bmatrix} rv - qw \\ pw - ru \\ qu - pv \end{bmatrix} + \frac{1}{m} \begin{pmatrix} f_x \\ f_y \\ f_z \end{pmatrix} \quad (2.11)$$

The aerodynamic forces acting on aircraft represent drag and similar forces in outdoor flight. In addition, since the project assumes that the quadcopter will fly indoors, it is assumed that in the ideal case, except for disturbances such as groundeffect, only the thrust affects the aircraft. Then each propellers of motors create aerodynamical thrust, upwards to actual body frame when angle of attacks of propellers are assumed zero. In

a situation where the quadcopter is level with the ground, with a roll and pitch angle of zero, the aerodynamic forces generated by the propellers will work to balance the weight of the quadcopter, causing it to hover, rise, or descend. As explained above, the net force is z-directed in the actual body frame of the drone:

$$\begin{pmatrix} 0 \\ 0 \\ f_z \end{pmatrix} - R_v^b \begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix} = m \left[\begin{pmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{pmatrix} + \begin{pmatrix} p \\ q \\ r \end{pmatrix} \times \begin{pmatrix} u \\ v \\ w \end{pmatrix} \right] \quad (2.12)$$

Then rewrite this equation, $\dot{V}_{CG/i}^b$ become:

$$\begin{pmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ f_z/m \end{pmatrix} - R_v^b \begin{pmatrix} 0 \\ 0 \\ g \end{pmatrix} - \left[\begin{pmatrix} p \\ q \\ r \end{pmatrix} \times \begin{pmatrix} u \\ v \\ w \end{pmatrix} \right] \quad (2.13)$$

This equation dictates how the velocity of the center of gravity of the quadcopter evolves in its body-fixed frame. In addition, this equation will be used in the state space model. To obtain another set of state space variables, it is necessary to project the vector onto the inertial frame in order to calculate the velocity within this coordinate system. This results call as kinematic equation of translational motion and given in the equation (2.5).

Concerning the equations of forces and their state variables, it is necessary to specify the form of the aerodynamical force generated by the propellers. Each propeller thrust given as in the k^b axis (angle of attack of propeller is assumed zero):

$$f^b = \sum_{n=1}^4 f_n^b = \begin{pmatrix} 0 \\ 0 \\ T_n \end{pmatrix}, n \text{ is the motor index} \quad (2.14)$$

Where T_n represents the upward thrust force generated by each propeller. This means that f_x^b and f_y^b are zero. The relationship between thrust produced by each motor and angular velocity is explained in the Motor Models section. And the result equation given in the Section 2.4.

2.3.2 Momentum Equations - Rotational Motion

These equations dictate the rotational dynamics of the quadcopter. Following the theorem of angular momentum.

$$\sum M = \frac{d}{dt_i} h \quad (2.15)$$

where h denotes the angular momentum around the center of gravity. Using coriolis equation:

$$\sum M^b = \left(\frac{d}{dt_b} h^b + \omega_{b/i}^b \times h^b \right) \quad (2.16)$$

For a rigid body, angular momentum is defined as the product of the inertia matrix J and the angular velocity vector $h^b \triangleq J\omega_{b/i}^b$. Then rearrange equation (2.16):

$$\sum M^b = (J^b \dot{\omega}_{b/i}^b + \omega_{b/i}^b \times J\omega_{b/i}^b) \quad (2.17)$$

Here J denotes the inertia matrix of the quadcopter. This Inertia Model explained in Section 3. But from the assumptions that the body of the quadcopter is symmetrical around all its axes, the inertia matrix has all off diagonal terms equal to zero. Then the inertia matrix J approximated in form of:

$$J = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix} \quad (2.18)$$

From last momentum equations $\dot{\omega}_{b/i}^b$:

$$\begin{pmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{pmatrix} = (J)^{-1} \left[\begin{pmatrix} M_x \\ M_y \\ M_z \end{pmatrix} - \begin{pmatrix} p \\ q \\ r \end{pmatrix} \times J \begin{pmatrix} p \\ q \\ r \end{pmatrix} \right] \quad (2.19)$$

To calculate the total momentum generated in the quadcopter system, it is important to know the rotation direction of each motor. The manufacturer of the Crazyflie 2.1 provides this information in Figure 6:

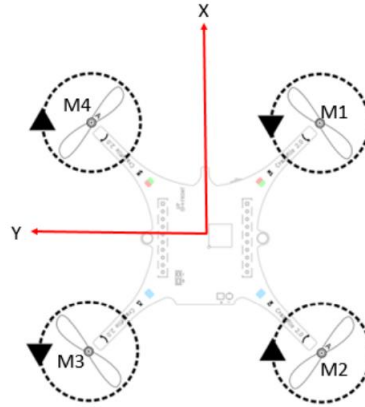


Figure 6 : “X” configuration and motor number of CrazyFlie2.1.

Mathematical modeling and controller designs made with X notation can be valid for any quadcopter, not just CrazyFlie 2.1. Then the expression become:

$$M = \sum_{n=1}^4 P_n \times f_n + \sum_{n=1}^4 \tau_n \quad (2.20)$$

Where P_n represents the position of each motor in the body-fixed frame and τ_n represents the induced momentum in the quadcopter's body generated by the n-th motor. If d denotes the distance from the center of gravity to the center of each motor:

$$P_1 = \begin{pmatrix} d / \sqrt{2} \\ -d / \sqrt{2} \\ 0 \end{pmatrix} \quad (2.21)$$

$$P_2 = \begin{pmatrix} -d / \sqrt{2} \\ -d / \sqrt{2} \\ 0 \end{pmatrix} \quad (2.22)$$

$$P_3 = \begin{pmatrix} -d / \sqrt{2} \\ d / \sqrt{2} \\ 0 \end{pmatrix} \quad (2.23)$$

$$P_4 = \begin{pmatrix} d / \sqrt{2} \\ d / \sqrt{2} \\ 0 \end{pmatrix} \quad (2.24)$$

Then Vector cross product of each motor become:

$$P_1 \times f_1 = \begin{pmatrix} -f_1(d / \sqrt{2}) \\ -f_1(d / \sqrt{2}) \\ 0 \end{pmatrix} \quad (2.25)$$

$$P_2 \times f_2 = \begin{pmatrix} -f_2(d / \sqrt{2}) \\ f_2(d / \sqrt{2}) \\ 0 \end{pmatrix} \quad (2.26)$$

$$P_3 \times f_3 = \begin{pmatrix} -f_3(d / \sqrt{2}) \\ f_3(d / \sqrt{2}) \\ 0 \end{pmatrix} \quad (2.27)$$

$$P_4 \times f_4 = \begin{pmatrix} f_4(d / \sqrt{2}) \\ -f_4(d / \sqrt{2}) \\ 0 \end{pmatrix} \quad (2.28)$$

The induced moments τ_n in the quadcopter system act only in the Z axis and have an opposite magnitude from the moment generated by each propeller. This is because the conservation of angular momentum requires that the total momentum in the system remains constant. In this case, the axis convention being used has the Z axis pointing upwards. Using the right-hand rule, we can determine that a clockwise spinning rotor generates a negative momentum (since the thumb points downward, in the opposite direction of the Z axis). As a result, the induced momentum will be positive. Then total induced moments vector can be shown as:

$$\tau^b = \sum_{n=1}^4 \begin{pmatrix} 0 \\ 0 \\ \tau_n \end{pmatrix}, n \text{ is the motor index} \quad (2.29)$$

The result of the force and angular momentum equations varies depending on how the engine is modeled. The closed form is given in this section and the results of these equations will be given in Section 2.4 according to the selected model.

2.4 Mathematical Model Parameters

Detailed system identification procedures are described in the doctoral thesis [3]. The result of the model is given here as a simple shortcut. In this section, most of the parameters used in the simulation environment setup and completion of the mathematical model will be given. Some specific details are given in Section 3.

2.4.1 Thrust Model

Two methods can be mentioned for modeling T_n , which represents the thrust produced by each motor in the equation given in 2.4. One of these methods can be defined by the relationship between motor thrust and motor angular velocity as specified in [3]. The equation corresponding to the reference taken from the article is as follows:

$$T_n = 1.2819 * 10^{-7} * \omega_n^2 + 7.23019 * 10^{-5} * \omega_n + 0.0095 \quad (2.30)$$

The other method is based on determining a coefficient instead of using all these equations. This coefficient is called the thrust coefficient and is expressed as C_T . The corresponding equation is as follows:

$$T_n = C_T \omega_n^2 \quad (2.31)$$

As a result, the result given in Equation 2.31 was used in determining the thrust produced in this study. Thus, it became easier to write the expressions in the rest of the equations. However, it is also thought that it would be more accurate to use the first method in future studies. The C_T value used for CrazyFlie 2.1 is given in Table 3.

Then total body force (without aerodynamic effects) can be summarized as:

$$f^b = \sum_{n=1}^4 f_n^b = \begin{pmatrix} 0 \\ 0 \\ C_T(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2) \end{pmatrix} \quad (2.32)$$

2.4.2 Torque

Two different methods can be used for the torque model as well. The first method involves using the mapping functions provided in [3]. The mapping function that transitions from thrust to motor torque is as follows.

$$\tau_n = 0.005964552 * f_n + 1.563383 * 10^{-5} \quad (2.33)$$

It seems possible to substitute two different equations for the thrust expression here. However, since these equations are derived using data fitting methods, the expression given in Equation 2.30 was also used during the fitting process. Therefore, substituting the result from Equation 2.31 here may lead to erroneous outcomes, albeit not significantly.

Another method is to model the induced torque using a coefficient. If f_n in the expression given in Equation 2.33 is replaced with Equation 2.30 or Equation 2.31, it can be observed that a function proportional to the square of the motor's angular velocity emerges. Consequently, the equation can be defined as follows:

$$\tau_n = (direction) \times C_D \omega_n^2 \quad (2.34)$$

The value of the induced torque varies as positive or negative depending on the motor's rotation direction. The C_D value is provided in Table 3.

When all the obtained results are substituted, the total body moment equation given in Equation 2.20 is derived as follows:

$$M = \begin{pmatrix} dC_T/\sqrt{2}(-\omega_1^2 - \omega_2^2 + \omega_3^2 + \omega_4^2) \\ dC_T/\sqrt{2}(-\omega_1^2 + \omega_2^2 + \omega_3^2 - \omega_4^2) \\ C_D(-\omega_1^2 + \omega_2^2 - \omega_3^2 + \omega_4^2) \end{pmatrix} \quad (2.35)$$

2.4.3 Result

The coefficients to be used in all the given equations are found as follows in the Table 3.

Table 3 : Model Parameters.

Parameter	Description	Value
m_{quad}	Mass of the quadcopter alone	28 [g]
d	Arm length	$39.73 \times 10^{-3} [m]$
r	Rotor radius	$23.1348 \times 10^{-3} [m]$
I_{xx}	Principal Moment of Inertia around x axis	$16.572 \times 10^{-6} [kg \times m^2]$
I_{yy}	Principal Moment of Inertia around y axis	$16.656 \times 10^{-6} [kg \times m^2]$
I_{zz}	Principal Moment of Inertia around z axis	$29.262 \times 10^{-6} [kg \times m^2]$
C_T	Thrust coefficient	$3.1582 \times 10^{-10} [N/rpm^2]$
C_D	Torque coefficient	$7.9379 \times 10^{-12} [N/rpm^2]$

Then for motor specific mapping commands can be given from [3] in the set of equation below.

32-bit PWM commands to thrust mapping:

$$f_n = 2.130295 \times 10^{-11} cmd_n^2 + 1.032633 \times 10^{-6} cmd_n + 5.484560 \times 10^{-4} \quad (2.36)$$

32-bit PWM commands to angular velocity of motor mapping:

$$\omega_n = 0.04076521 * cmd_n + 380.8359 \quad (2.37)$$

Thrust to torque mapping:

$$\tau_n = 0.005964552 * f_n + 1.563383 * 10^{-5} \quad (2.38)$$

For detailed modelling of quadcopter also can look [13].

2.5 State Space Model Representation

The State Space model is a model that makes sense of the system by establishing a relationship between the determined state variables and their derivatives, and its equations are as follows:

$$\dot{x}(t) = A(t) x(t) + B(t) u(t) \quad (2.39)$$

$$\dot{y}(t) = C(t) x(t) + D(t) u(t) \quad (2.40)$$

The equations (2.30) and (2.31) called general state space equation. This equation describes a linear time-varying system, where $x(t)$ is the vector of states, $y(t)$ is the output vector, and $u(t)$ is the input vector. The system is represented in state space form, where A , B , C , and D are matrices that determine the behavior of the system. This particular representation is called a linear time-invariant system, which means that the matrices are static and do not change over time. This can be contrasted with a linear time-varying system, where the matrices can change over time. Where Δ linearized result:

$$\Delta \dot{x} = A \Delta x + B \Delta u \quad (2.41)$$

$$\Delta \dot{y} = C \Delta x + D \Delta u \quad (2.42)$$

The first question to be considered in the linearization process is for which operating point (operating point) this linearization will be performed. This has to be associated with an equilibrium point in which the states of the system do not change over time. In the case of the quadcopter, the common trim trajectory is the hover. Then equilibrium states can be defined:

$$x_e = (x_e \quad y_e \quad z_e \quad \psi_e \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0)^T \quad (2.43)$$

In addition, to keep the drone parallel to the ground at a constant height, each motor must rotate at the same angular velocity to create the same thrust. We can easily say this thanks to the symmetry assumption we made:

$$C_T(\omega_{e1}^2 + \omega_{e2}^2 + \omega_{e3}^2 + \omega_{e4}^2) = mg \quad (2.44)$$

$$\omega_{e1} = \omega_{e2} = \omega_{e3} = \omega_{e4} \quad (2.45)$$

Then, for the ideal hover position condition, the angular velocity of each motor is found as:

$$\omega_e = \sqrt{mg / 4C_T} = 16073 \text{ [rpm]} \quad (2.46)$$

Then the input vector become equal each other:

$$u_e = (\omega_e \quad \omega_e \quad \omega_e \quad \omega_e) \quad (2.47)$$

After this stage, the linearized state space model can be made with the taylor expansion method. It should be noted that in the hover condition, the body frame and the inertial overlap each other and some such simplifications can be made within the model. However, for the sake of brevity, since it has not been used in detail in the project yet, detailed calculations are not included. For more detailed calculations, please see [4]. Again, as a result of this linearization process, the system can be considered independently as a vertical and lateral subsystem around the hover condition.

3. SIMULATION

In this section, how the mathematical model, which was explained in detail in Section 2, was established in the Matlab/Simulink environment with the help of the SimScape library, how the model parameters were integrated in the simulation and the block structures will be explained. In the Figure 7 general model scheme given.

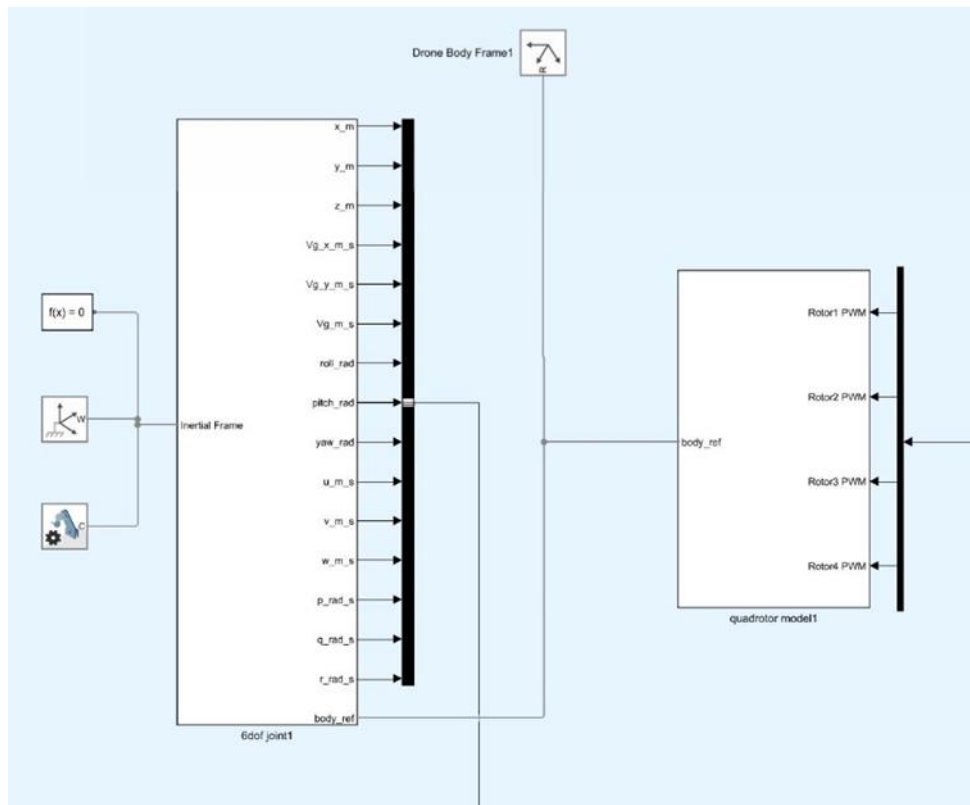


Figure 7 : Simulink 6DOF joint and quadrotor model.

The 6DOF joint block contains operations related to aircraft estimates and coordinate frames. The quadrotor model block contains sections such as quadrotor parameters, drag coefficient, inertia and motor model.

3.1 Quadrotor Model

The general quadrotor simulink diagram is as seen Figure 8. Two main points stand out here. Thrust and Rotation calculation with equations calculated by means of the Motor and Drag Coefficient model, and then these values, which are converted into Physical Signal, are given to the drone Rotor Models. After this process, the necessary

force and torque are created on the drone body. These connections are then integrated into the body.

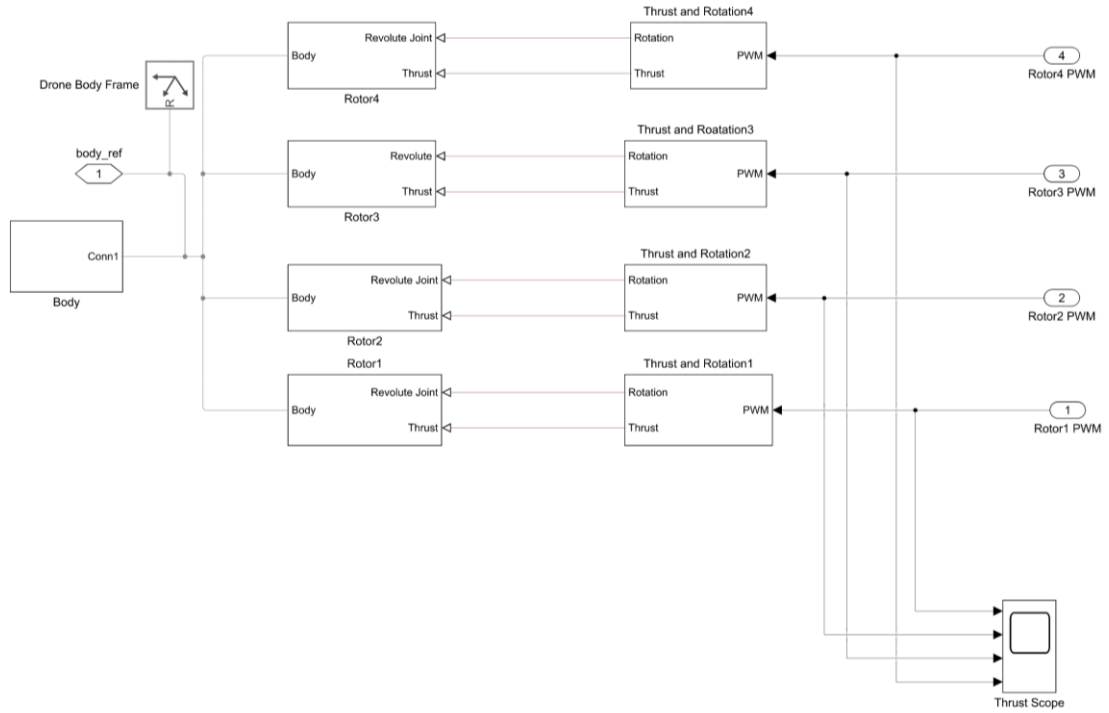


Figure 8 : Quadrotor model subsystem block.

The most important point to be considered here is the axis definitions and the direction of the force. Another important issue is the necessity of passing through the integral block in order to convert the angular velocity to physical rotation. It also acts as a kind of filter.

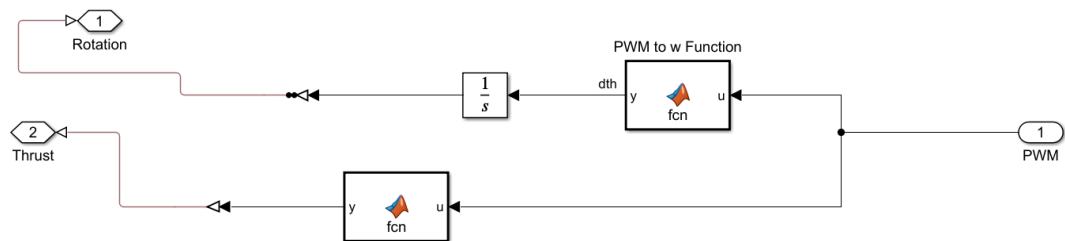


Figure 9 : CW direction motor.

The transfer function and mapping functions given in the Section 2.4 could also be used for the thrust calculation. For simplicity command pwm to thrust mapping function implemented.

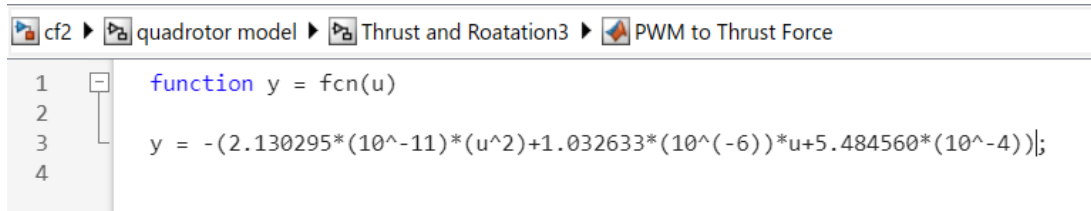


Figure 10 : PWM To Thrust Force Function Block.

There are two types of inputs that originate from the motor and are applied to the quadrotor body. One of these is the moment caused by the rotation of the engine, and the other is the thrust force generated by the propeller. SimScape allows torque calculation in the revolute joint from input motion. For this reason, motion was produced using PWM to ω mapping.

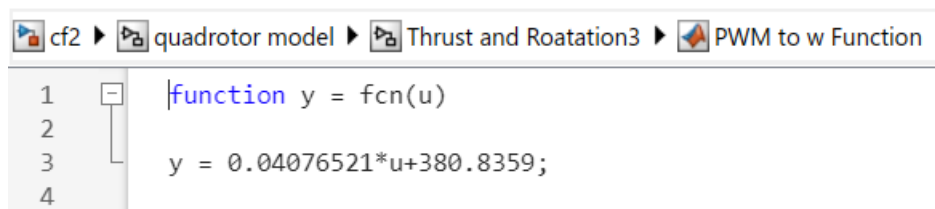


Figure 11 : PWM To ω Function Block.

As mentioned above, it is necessary to give the direction of rotation so that the revolute joint can calculate the moment from the input motion. For this reason, the rotation direction in the CCW motor is provided by multiplying by -1.

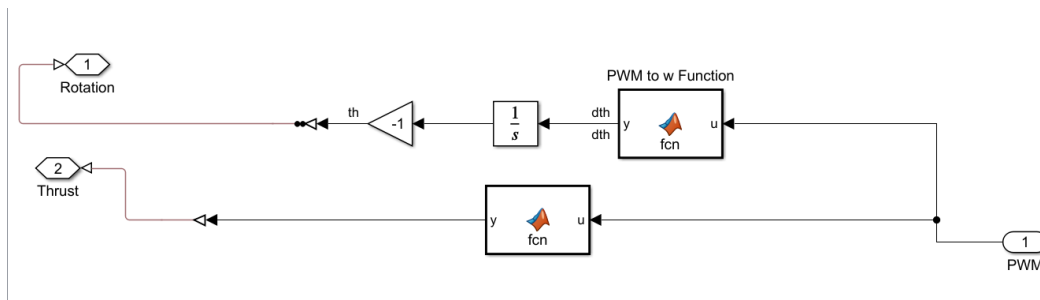


Figure 12 : CCW direction motor.

Since there is an output of 0 PWM in the PWM TO W function, the function must output above a certain pwm value. This threshold value is set at 1000PWM. Also if we don't use this system is work properly.

The three most important points used in the Rotor Joints are: The first is the Motor Thrust Block. This block implements the force to be applied to the propeller and body through SimScape. Another important block is the Revolute Joint block. This block

calculates the Torque on the rotor from the Rotation movement given as input. The rotor block given in the Figure 13.

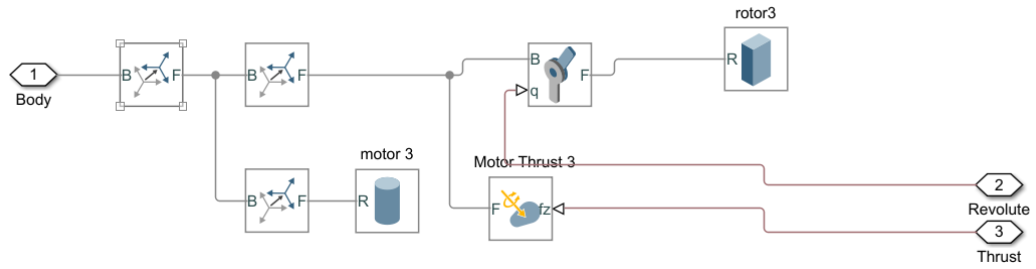


Figure 13 : Rotor Simscape Implemented Model.

And some other parameters using in the implementation of models are:

$$g = 9.80665 \text{ m/s}^2 \quad (3.48)$$

$$\text{Hlaf Length of Body } \frac{d}{2} = 4.65 \text{ cm} \quad (3.49)$$

$$\text{Mass of Propeller} = 0.0005 \text{ kg} \quad (3.50)$$

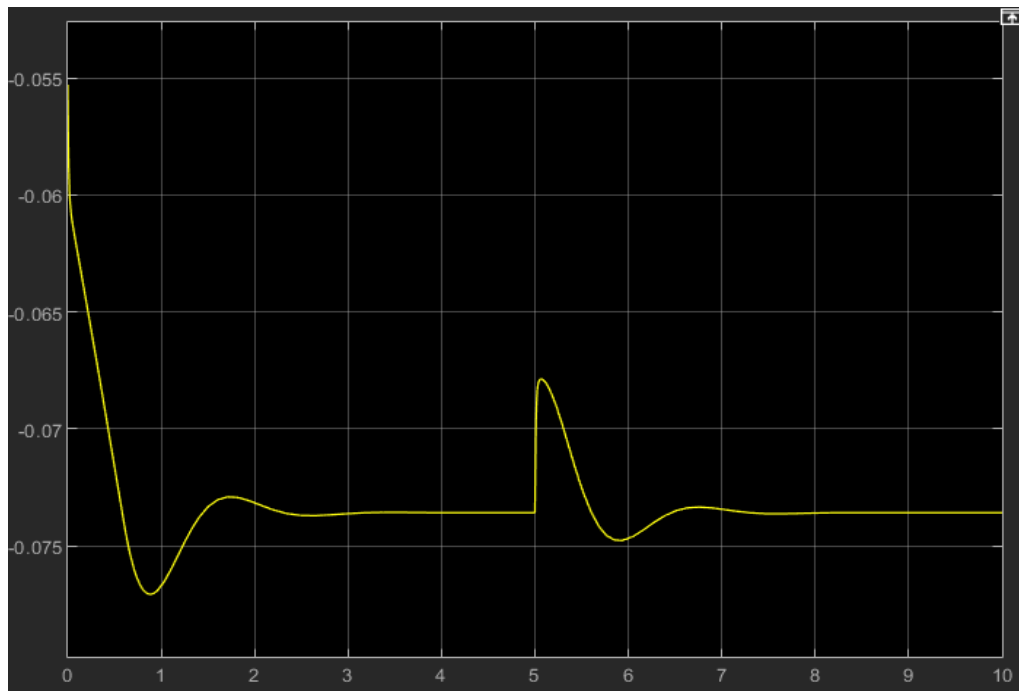


Figure 14 : Sample generated thrust, CCW direction.

3.1.1 Propeller Inertia Calculation

In the Simulink model, the propeller model that we will put in the Simscape library is considered as a rectangular prism by approximation. The CAD model of propeller shown in Figure 15.

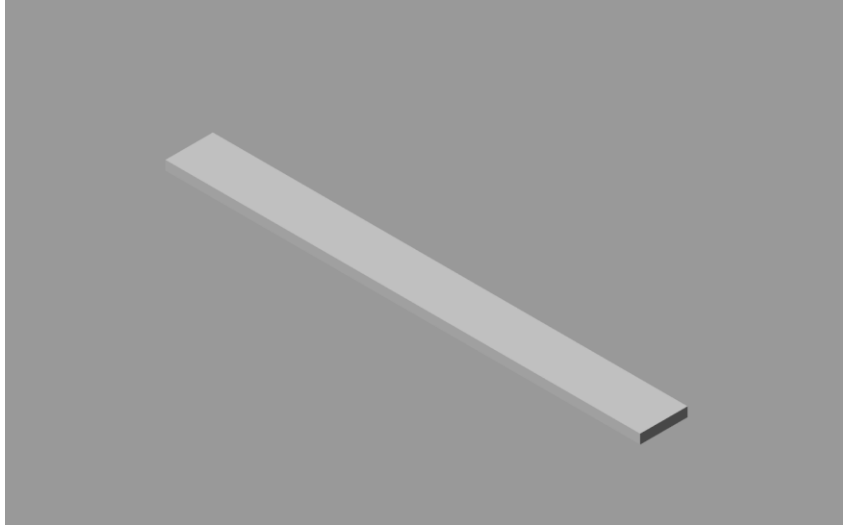


Figure 15 : Rotor CAD model.

According to the Figure 16 and considering the rotation direction of the propeller, it will be sufficient to add only the Inertia in the Z axis to the model.

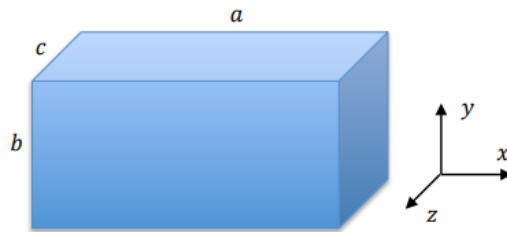


Figure 16 : Rotor CAD model.

The calculation is as follows. Mass is 0.0005kg:

$$I_{zz,Prop} = m \left(\frac{a^2}{12} + \frac{b^2}{12} \right) = 9.254166 * 10^{-8}, \quad a = 0.04, b = 0.014 \quad (3.51)$$

3.1.2 Damping Coefficient of Rotor

A damping mechanic is required to establish the motion model of the rotor. This models the energy loss in the rotor. For this reason, a dumping coefficient is required

for the Revolute Joint block under the SimScape library. No value was found as a reference source when selecting this coefficient. For this reason, this value intuitively:

$$b = 3.5077 * 10^{-10} \quad (3.52)$$

3.1.3 Drag Coefficient of Quadrotor

The drag coefficients given to us as the results of the experiment, our thrust and so on. In order to model the friction loss of the drone in the SimScape model, an average value should be obtained. In the simulation intuitively appriximated value is used:

$$b = 1 * 10^{-9} \quad (3.53)$$

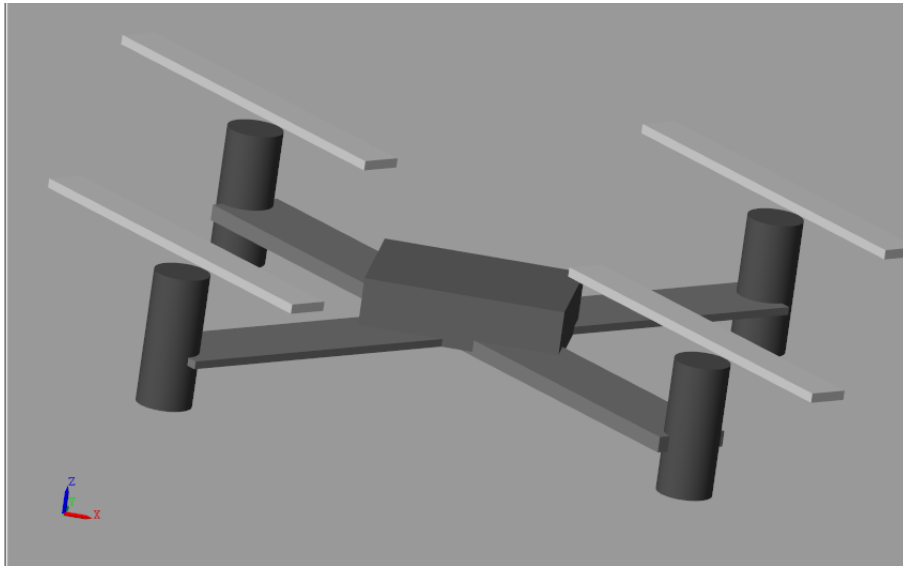


Figure 17 : Resultant SimScape model.

3.2 6DOF Joint and State Estimation

In this section, we will discuss how 6DOF equations and aircraft prediction values are obtained. This block, which has been implemented based on the logic of the coordinates frames mentioned in the Section 2, calculates where the drone is according to the intertial coordinate frame and calculates revolute joints operation and give estimation of attitude, attitude rate, velocity of quadrotor.

In the Figure 18 can see the internal structure of the 6DOF-Joint block. The mathematical operations performed in this internal structure can be defined as follows. As explained in the mathematical modeling section of this study, translations and rotations between frames of reference are made via the simscape library. The body

frame axis is created in the Z-X-Y notation of rotation, respectively from vehicle frame. Vehicle frame is obtained from translation of inertial frame. To define translation and rotation respectively, $Trans_{inertial\ frame\ (x,y,z)}$, $Rot_{vehicle,\psi}$, $Rot_{vehicle-1,\phi}$, $Rot_{vehicle-2,\theta}$, refer Table 1.

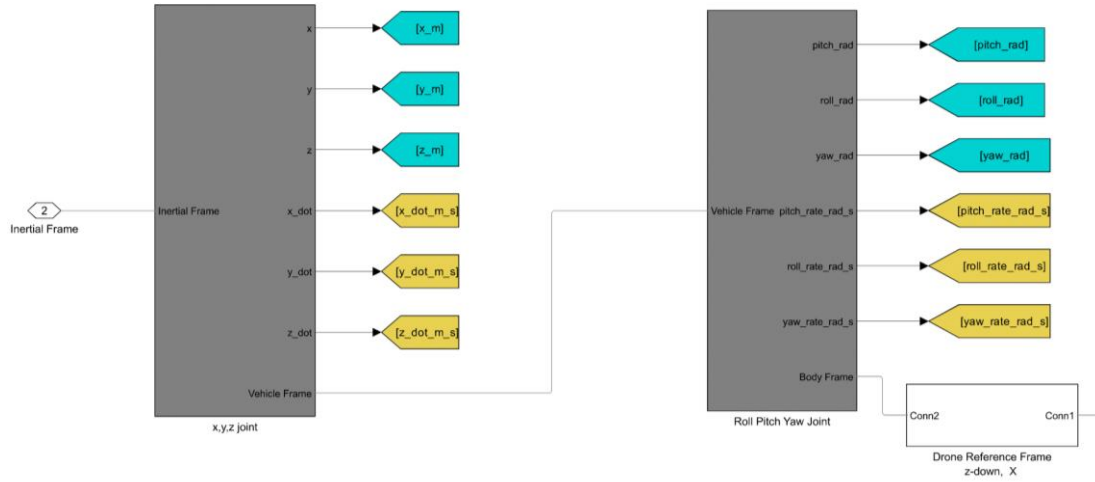


Figure 18 : 6DOF joint first section.

These translation and rotation operations are implemented in the x, y, z joint block and the Roll Pitch Yaw Joint block, respectively. Thus, all reference systems defined in the mathematical modeling section and the values defined in these reference systems can be obtained using output of this blocks.

In order to fully perform the maneuvering task in simulation, the dynamics of the quadcopter must be modeled as close to reality as possible and the required estimation values for the algorithm and controllers must be obtained using this model. The obtained estimation values must be able to be obtained in real life with sensors and estimation algorithms. Depending on the platform (open source autopilots, etc.), the required estimation values may also need to be implemented in necessary cases. For this purpose, the state variables given in the Table 2 and required for the closest mathematical modeling of the real quadcopter in the simulation environment are included.

Values such as roll, pitch, yaw angle can be obtained from the values given in this table by using the joint blocks of the Simscape library. The values that can be obtained by using the blocks are as follows:

- X,Y,Z Position wrt Inertial Frame
- Roll Angle (Which is intermediate frame rotations angle)

- Pitch Angle (Which is intermediate frame rotations angle)
- Yaw Angle (Which is intermediate frame rotations angle)

In order to obtain the body angular velocity and body velocity values, the equations defined in the mathematical modeling section of the report must be reviewed again.

In the given relation $\dot{p}_{CG/i}^i$ corresponds to the aircraft ground speed defined in the inertial frame. $V_{CG/i}^b$ corresponds to the ground speed defined in the body frame. As a result, the rotation matrix is needed in the relation defined in the following equation. However, in the following relation $\dot{p}_{CG/i}^i$ is obtained by applying body to inertial frame rotation to $V_{CG/i}^b$. Rearrange this equation which multiply bothside $(R_i^b)^{-1} = R_i^b = (R_b^i)^T$. Or simply thinking, u,v,w is definition of ground speed vector into the body frame, then simple rotation can be used inertial frame to body frame:

$$\dot{p}_{CG/i}^b = V_{CG/i}^b = (R_i^b) \dot{p}_{CG/i}^i \quad (3.54)$$

Which R_i^b given in the mathematical modeling section. Also this matrix depends on the roll pitch and yaw intermediate axis angle. Using same approach, result of this notation, the body angular velocity vector is defined as follows, as stated in the mathematical modeling equation (2.7).

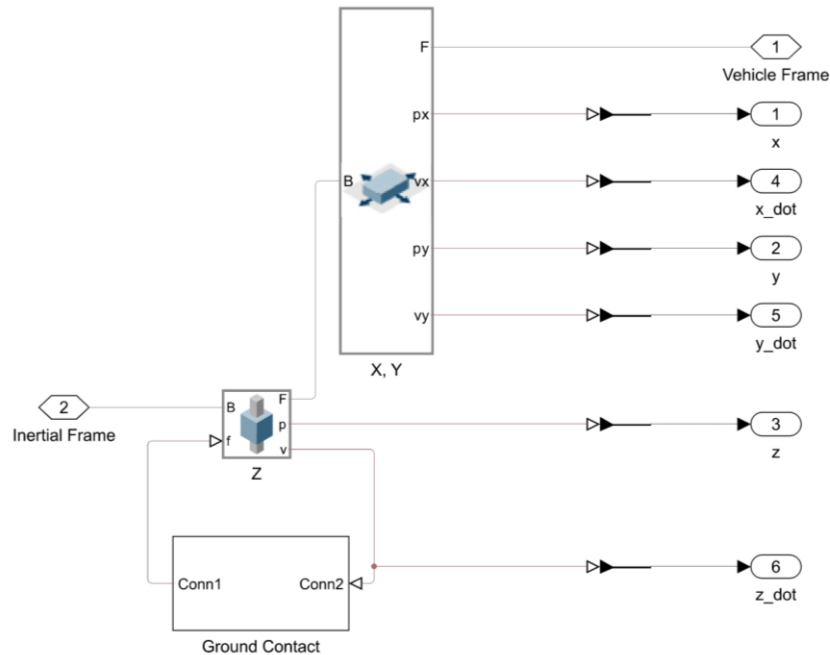


Figure 19 : 6DOF joint translational joint.

After obtaining given first 6 state variables using Simscape blocks this mathematical relationships can be used for estimation values computations.

As a result, both the body frame is obtained as a result of the rotations made and it is possible to obtain Euler angles and Euler angle rates thanks to the joints that can be defined in Simscape. The Revolute Joint block offers the opportunity to measure up to the acceleration for the value it corresponds to. If necessary, these values can be obtained by editing the block parameters.

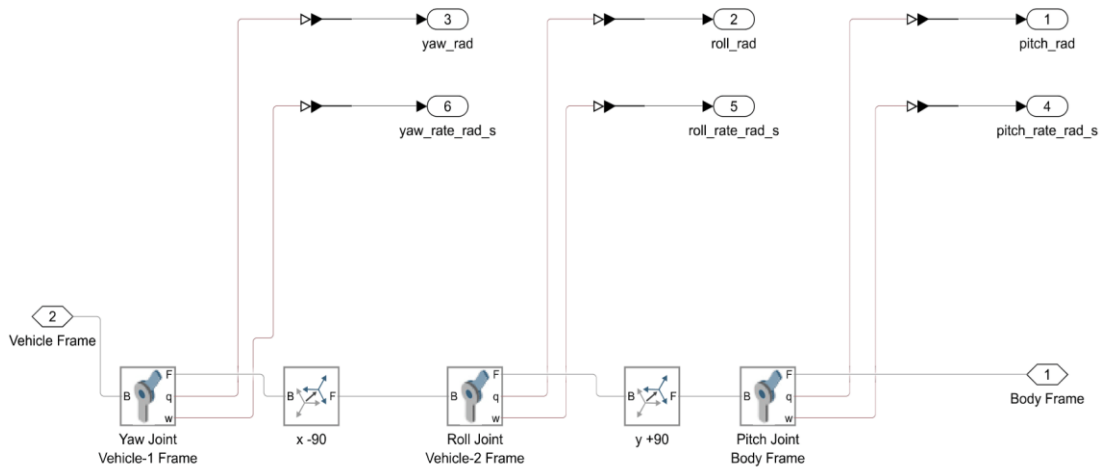


Figure 20 : 6DOF revolute joint.

After the frame of references for the realization of a drone model and the world are created in this way in Simulink, the required estimated values can be obtained. The trajectory generation algorithm and the controller to be designed for the flip maneuver aimed to be realized in this study will also need the angular velocity values of the body. For this reason, the p, q, r values specified in the mathematical modeling section are needed. Model equations can be used to obtain these values.

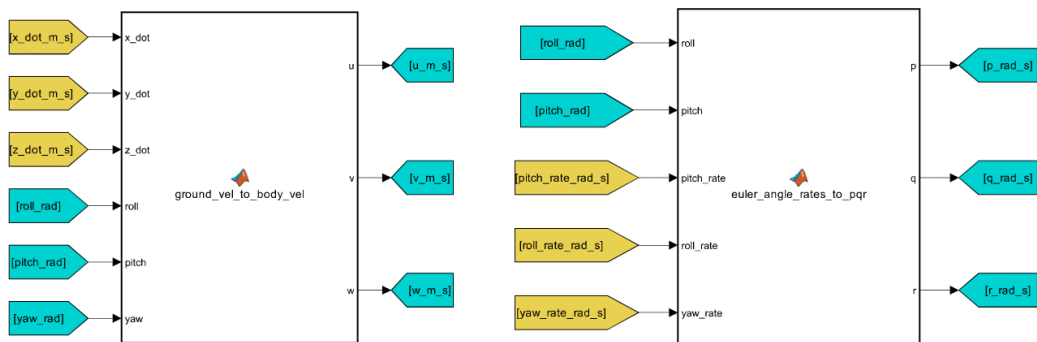


Figure 21 : 6DOF revolute joint.

It is seen in the model that there is a relationship between euler angle rates and body angular velocity. The reason for this relationship is that euler angle rates are not directly equal to the angular velocity values in the body axis. Because there are intermediate frames and a transformation is applied in order. Only in the case where roll, pitch, yaw angles are zero, euler angle rates are equal to body angular velocity.

Using this equation, p,q,r values can be observed on Simulink. As a result, the estimated values obtained for the zero reference are observed as follows:

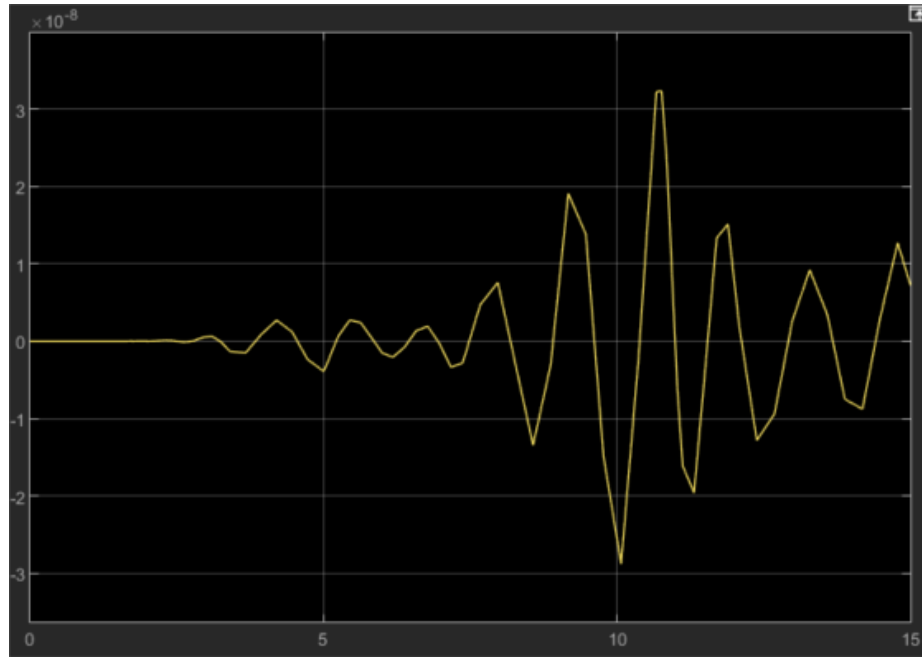


Figure 22 : “p” estimate.

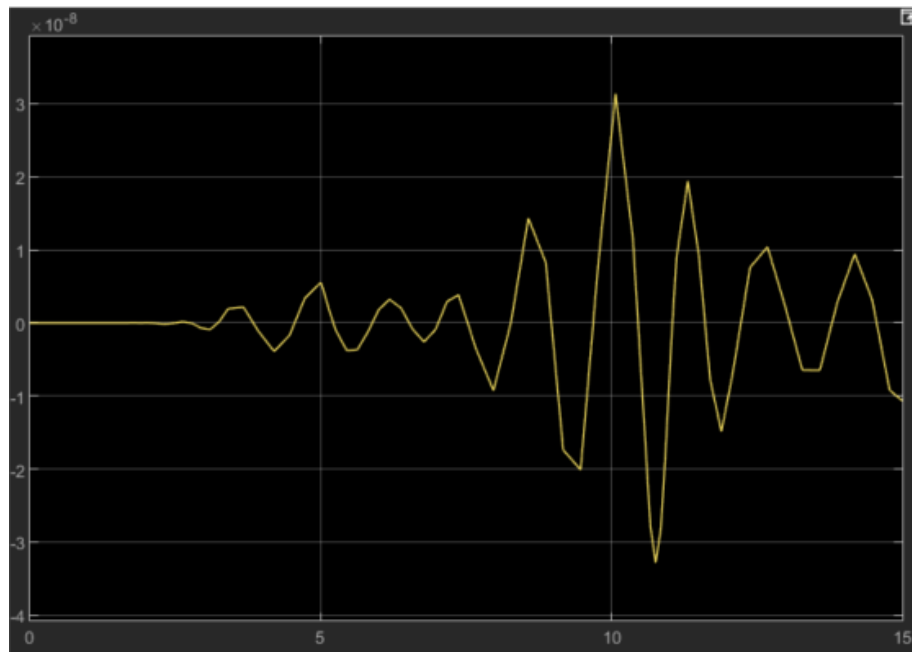


Figure 23 : “q” estimate.

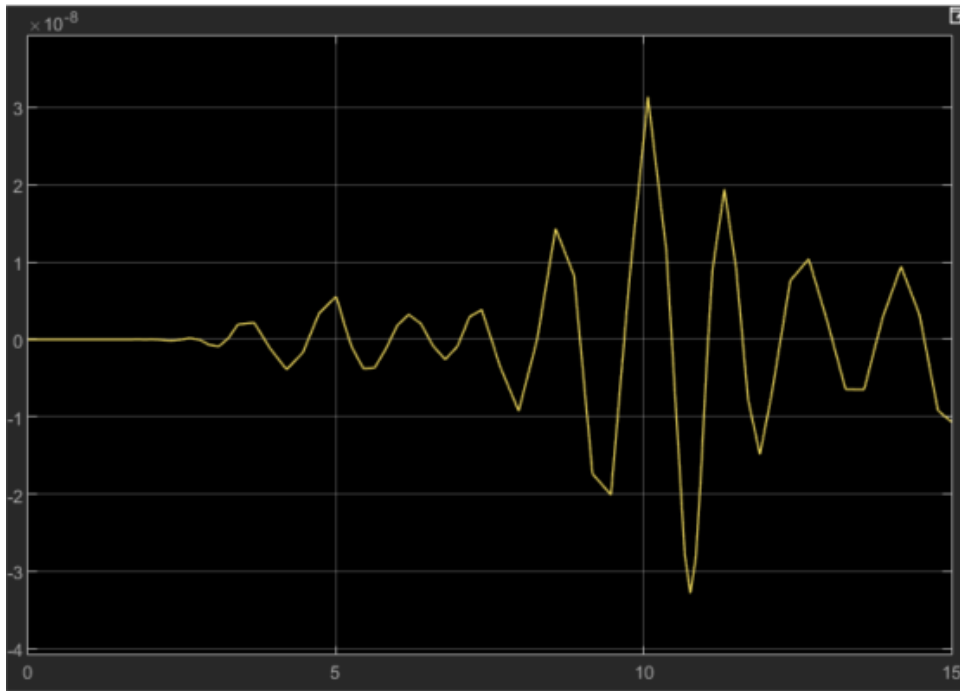


Figure 24 : “r” estimate.

Since the estimate value is in the region of 10^{-8} to 10^{-20} it is close to zero. However, there is a possibility that the estimate may need to be filtered for use in controllers. It is envisaged that the filtering process will be carried out according to the observed signal structure.

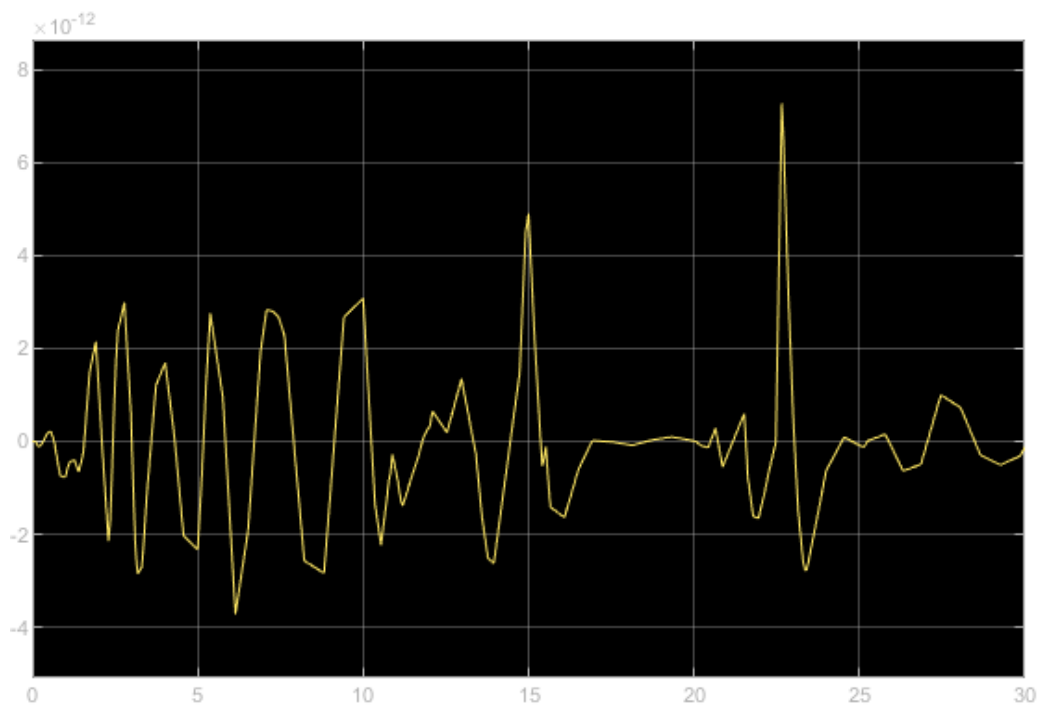


Figure 25 : “u” estimate.

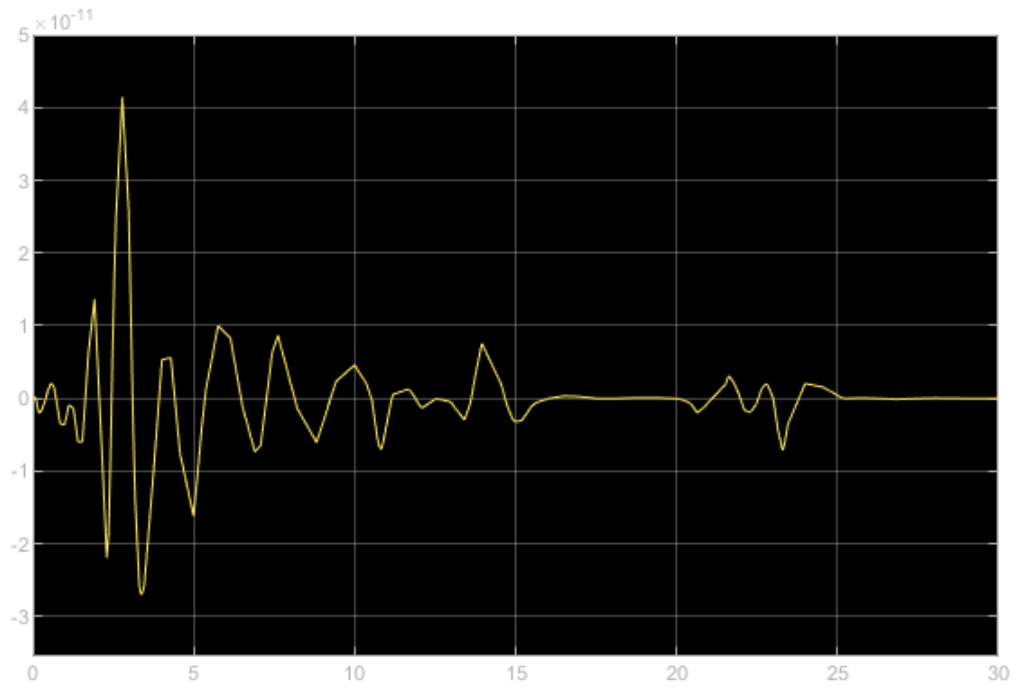


Figure 26 : “v” estimate.

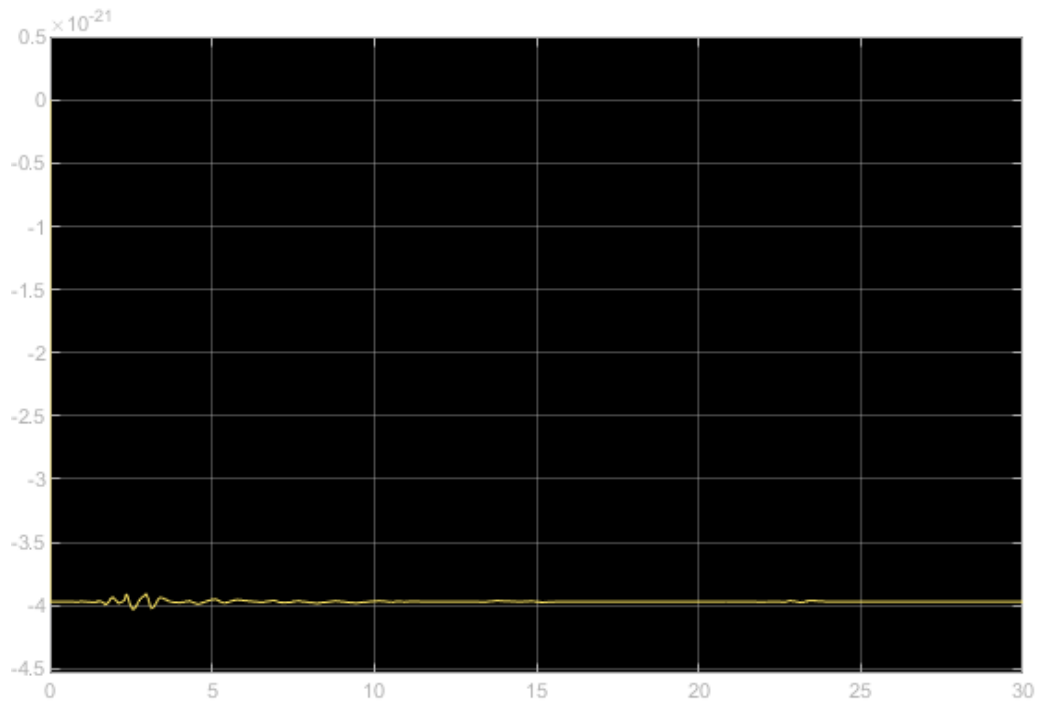


Figure 27 : “w” estimate.

As a result, all the aircraft estimations produced in the simulation are presented in Table 4.

Finally, it should be mentioned that the battery model was not added to the simulation. For this reason, assuming that the battery compensation algorithms in the official

firmware work well enough, it is predicted that the difference between the simulation result and the real test result will not increase too much.

Table 4 : State variables for MAV equations of motion.

Estimation	Description
x	X position of CoG in the inertial frame
y	Y position of CoG in the inertial frame
z	Z position of CoG in the inertial frame
V_{gx}	Ground velocity of CoG in the inertial frame X direction.
V_{gy}	Ground velocity of CoG in the inertial frame Y direction.
V_{gz}	Ground velocity of CoG in the inertial frame Z direction.
ϕ	Roll angle
θ	Pitch Angle
ψ	Yaw Angle
u	Body frame velocity measured along i^b in F^b
v	Body frame velocity measured along j^b in F^b
w	Body frame velocity measured along k^b in F^b
p	Angular velocity measured along i^b in F^b
q	Angular velocity measured along j^b in F^b
r	Angular velocity measured along k^b in F^b

4. CONTROLLER DESIGN

In this section, after the modeling of the selected platform and the creation of the simulation environment, the design and implementation of the controller, which is the official stepwise PID controller scheme that provides position control and stabilization of the aircraft, will be started. General scheme of this official controller structure given in Figure 28.

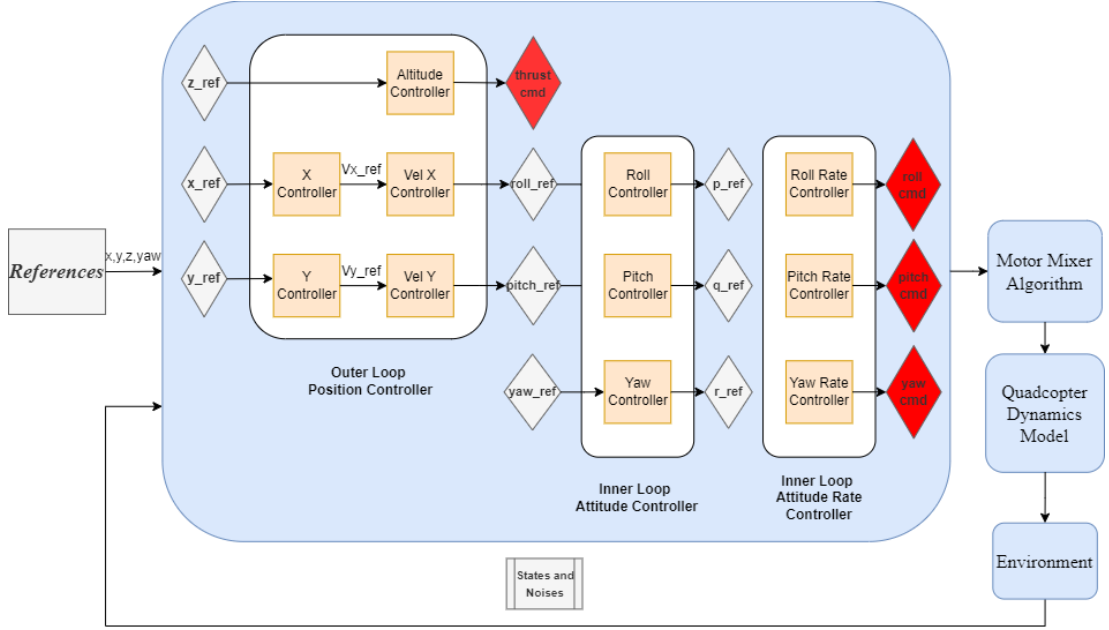


Figure 28 : Cascaded PID controller.

This position controller structure will ensure that the aircraft follows the reference x , y , z , yaw trajectory in the first stage. However, in order to implement aggressive maneuvers, it is also necessary to consider the angular and linear acceleration controller structures. The rule that the primary reason for position is speed, which is one of the first lessons taught by the dynamics title, should not be forgotten and trajectory planning should be evaluated under this title. It is thought that this controller will not be sufficient in aggressive trajectories, i.e. in flights that will take place within the feasibility limits of the high-acceleration vehicle, and its analyses will be carried out under this title.

In given structure, the reference block is block functions as the input for the entire system, defining a trajectory in the x , y , z , and yaw positions. Currently, the commands sent by this block are simply constants or signal inputs such as sinusoids, random signals, ramps, etc. It serves as input commands for the controller. In the controller

design phase, the inputs and outputs of the system become important. It also determines what path you want to control. Hover Control will be made in our system and our inputs are motor 16-bit PWMs (2^{16}). As output, the x, y, z, roll, pitch, yaw states of the drone are taken. One of the assumptions made when starting the business is that we are flying with small roll and pitch angles. In this way, thrust will only affect altitude.

The firmware of the Crazyflie incorporates a dual cascaded PID control system to manage the pitch and roll angles. In analyzing a cascaded control arrangement, we can break down the architecture into inner and outer control loops. The outer loop governs the inner loop, which in turn regulates the system's plant. A fundamental principle in cascaded setups is that the inner loop should operate at a higher rate than the outer loop. Ideally, the output of the inner loop should stabilize before the outer loop modifies the setpoint conveyed to it. If the inner loop's response is slower than required or if the outer loop operates too swiftly, synchronization issues may arise between the two controllers. In the given design, it has been observed that the inner loop controllers, attitude and attitude rate controllers operate at 500Hz.

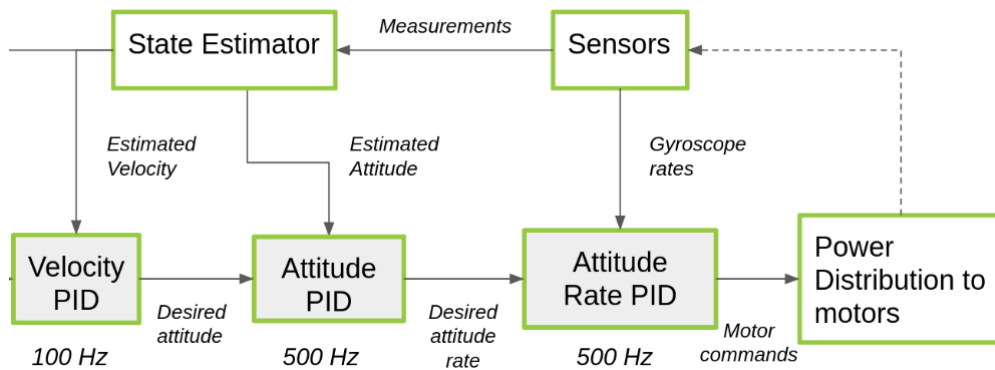


Figure 29 : Block Diagram of Crazyflie 2.1 Firmware Controller Scheme.

These attitude controllers generate the reference roll-pitch-yaw speed (change) with the PID applied to the roll-pitch-yaw positions that should be with the general logic.

4.1 Outer Loop Position Controller

In this section, the outer loop position controller structure and how it works in detail will be discussed. Outer loop position controllers can be used in the initial phase for waypoint navigation or trajectory tracking. By observing its performance during

aggressive maneuvers, its disadvantages can be easily observed and inferences can be made in the simulation environment.

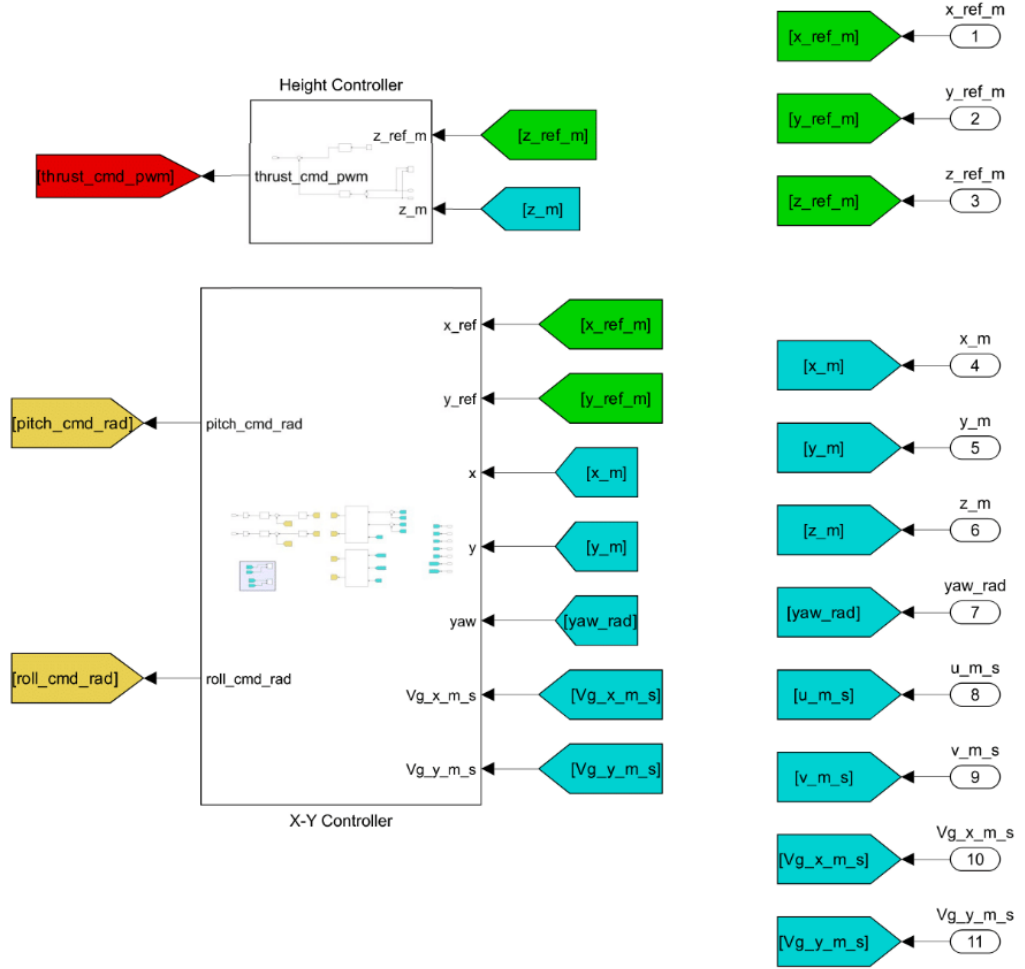


Figure 30 : Outer loop position controller structure.

Outer loop position controllers take x , y , z reference positions as input. As output, they produce commands are ϕ and θ . This is made possible by the dynamics of the quadcopter. Because when the quadcopter wants to change its position in the positive X direction, it needs to make a positive ϕ angle. Similarly, when the aircraft wants to change its position in the positive Y direction, it needs to make a negative θ angle. In this way, a reference pitch and roll angle is produced as the output of the outer loop cascaded position controllers. Before going into detailed explanation, the following controller structures are found under the title of outer loop position controllers:

1. An altitude controller whose output is the thrust required to maintain a certain position in Z . (Note that it is independent from X - Y behavior. However, for

aggressive flight maneuvers, they must be coupled or controlled by considering all motions together.)

2. An X-Y position controller whose outputs are the required roll and pitch angles.

4.1.1 Altitude Controller

First of all, the height PID controller included in the normal firmware was implemented and it was checked whether the simulation was performed correctly or not.

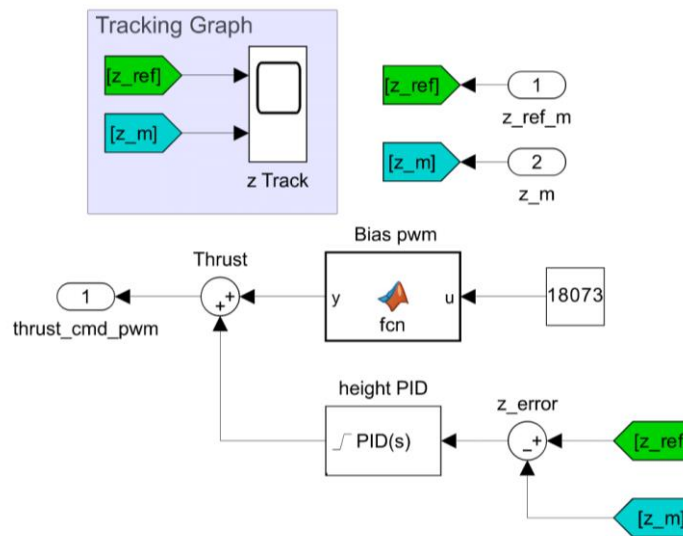


Figure 31 : Altitude PID controller.

This block, which is a simple PID controller, takes the height error and produces Thrust output with the PID controller. But this alone is not enough value. Therefore, a bias must be added. In the equations given in the Section 2, it is seen that the drone's motor should be at 16073 RPM in the equilibrium state. However, as a result of observations, since the model implemented on Simulink is an approximation, the 16073 RPM value is low and has been updated with the 17073 RPM value. This difference in value may be related to the error of the C_T coefficient used in the study taken as reference. It is obvious that this method is not correct and is not based on mathematics. However, since the controller side has approximated the result and there is no time to update the model, this estimate is continued. No changes have been made on the model side. The right way is to find the reason for this difference and correct it. To convert this value to PWM, the following equation is used and bias PWM is obtained. The input

command PWM to angular velocity of motor mapping given in Equation 2.37. is found using. Respectively:

$$\omega = (RPM \times 60) / 2\pi \quad (4.55)$$

$$\omega = 0.04076521 * cmd + 380.8359 \quad (4.56)$$

$$cmd = \frac{(\omega - 380.8359)}{0.04076521} \quad (4.57)$$

The controller is a PID controller and its parameters are given in Table 5:

Table 5 : Altitude PID controller parameters.

Parameter	Value
<i>PID_ALT_KP</i>	11000
<i>PID_ALT_KI</i>	3500
<i>PID_ALT_KD</i>	9000
<i>OUTPUT_LIM_UP</i>	15000
<i>OUTPUT_LIM_DOWN</i>	-20000

4.1.2 X-Y Controller

Here is the general structure of X-Y controller in Figure 32:

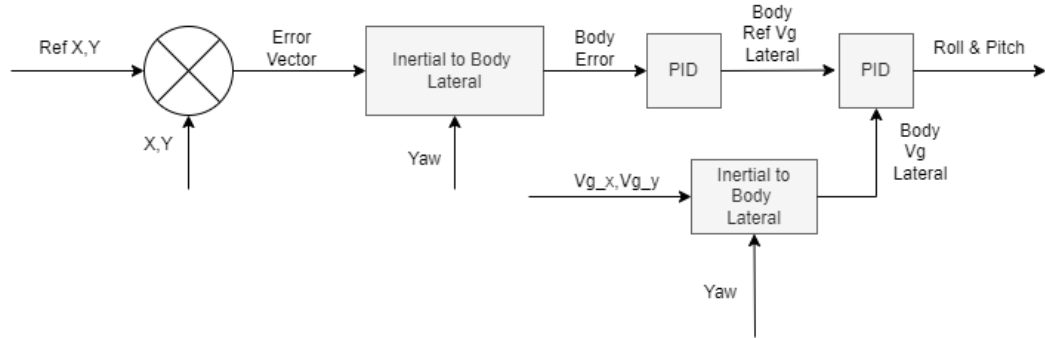


Figure 32 : X-Y position controller.

The controller's purpose is to direct the quadcopter's horizontal movement by computing the necessary roll and pitch angles, which are then fed to the onboard Attitude Controller. Since this position controller's outputs are roll and pitch angles that define the transformation between Vehicle Frames, error calculations are performed on the body frame. In other words, the error calculation is made in the inertial frame and these error values are projected onto the body frame. This projection process is only performed in the horizontal plane. Although it is not a general method,

controller design can be done as an approximation in horizontal and vertical planes specifically for quadcopters. This is shown more clearly in Figure 33.

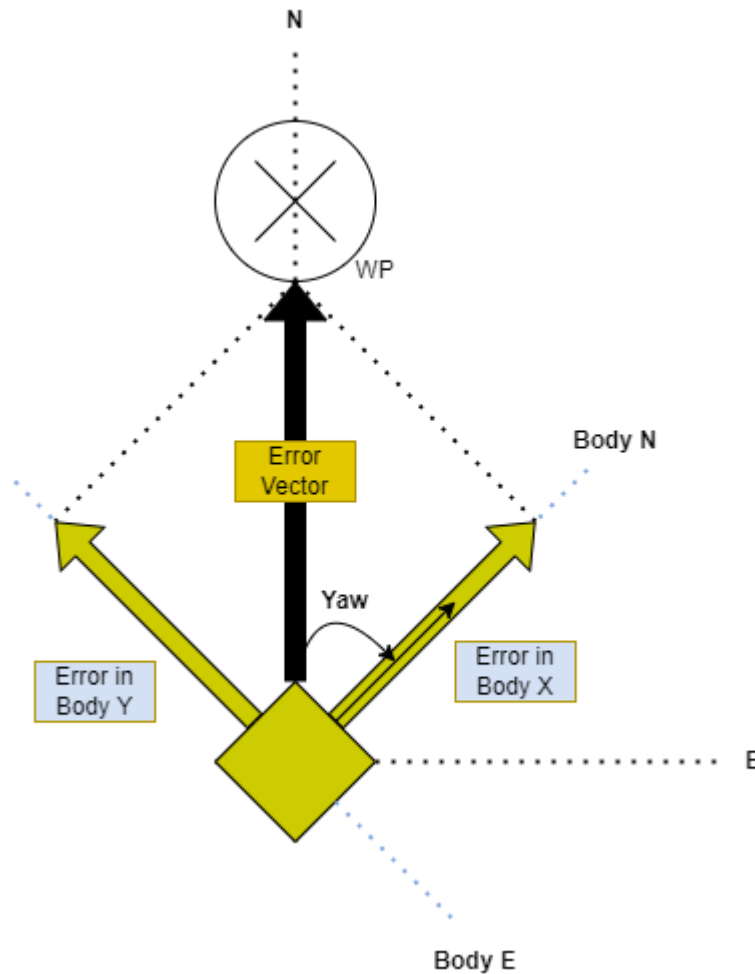


Figure 33 : Position body error logic.

The logic of this process can be explained as follows: The quadcopter makes a pitch rotation in the direction of its heading. Perpendicular to this at 90 degrees, it makes a roll rotation, and the desired motion vector is obtained through the combination of these two. To obtain this vector, the controller calculates the desired position vector laterally in the body axis. Thus, error values are obtained in the roll and pitch axes. This way, the desired combined motion can be achieved by directly generating roll and pitch inputs.

As a result, respectively:

- Define the error vector horizontally in the body frame.
- To zero the defined error vector, a pitch command is issued according to the magnitude on the Body X axis..

- To zero the defined error vector, a roll command is issued according to the magnitude on the Body Y axis..

This process is carried out as follows:

$$\begin{pmatrix} x_e^b \\ y_e^b \end{pmatrix} = \begin{pmatrix} x_e \\ y_e \end{pmatrix} \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \quad (4.58)$$

The controller block where these operations are performed is shown in detail in Figure 34:

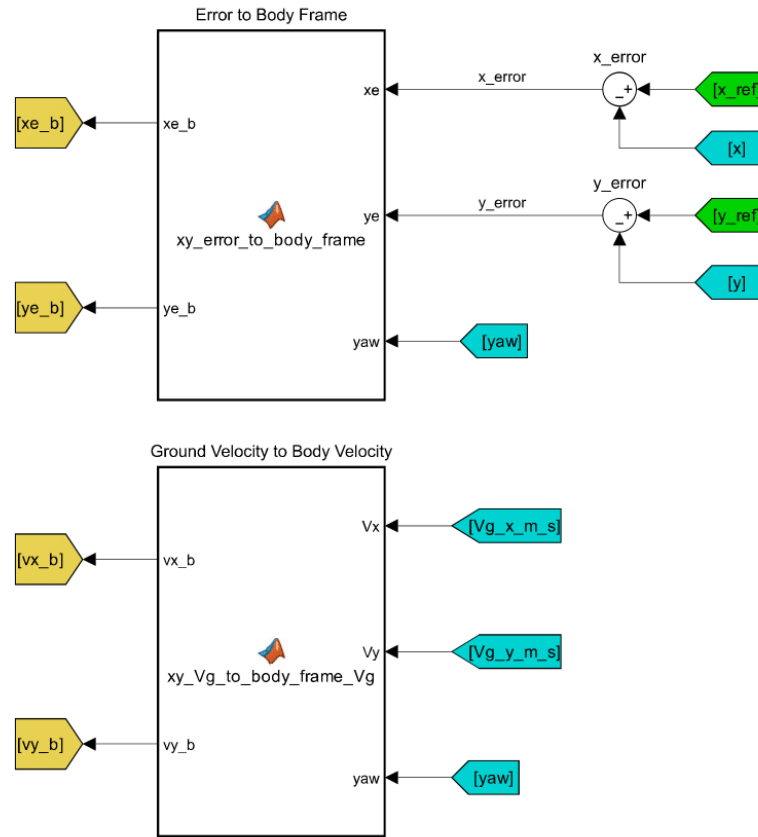


Figure 34 : Position body error logic.

In the second stage, cascaded position and speed controllers are operated. Their block diagram is shown in Figure 35.

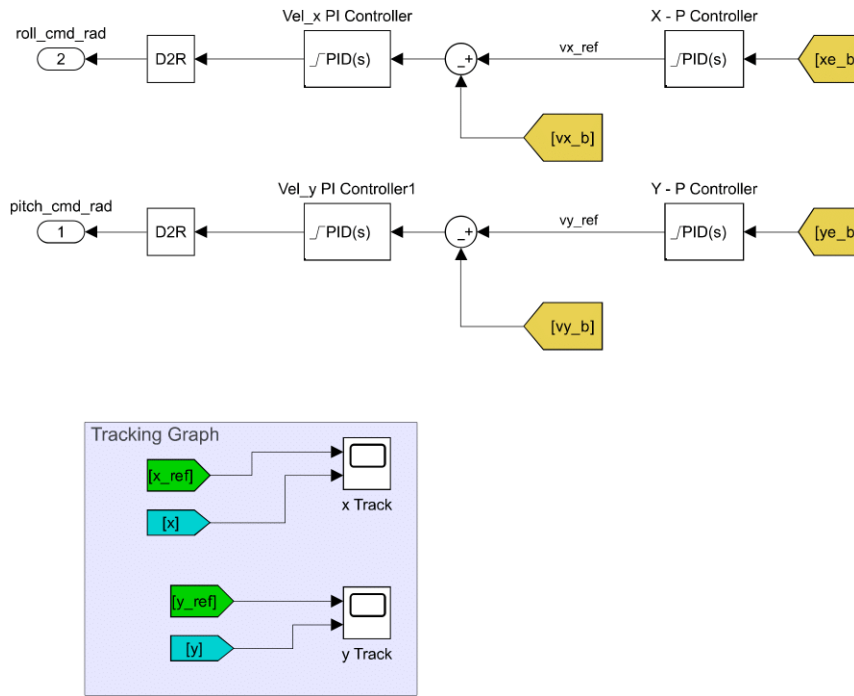


Figure 35 : Position body error logic.

Finally, the parameters of the controllers are given in Table 6, Table 7, Table 8 and Table 9 below.

$$OUTPUT\ LIMIT = MAX_VAL * Overhead \quad (3.59)$$

Table 6 : Position X, P Controller.

Parameter	Value
$PID_POS_X_KP$	2.0
$PID_POS_X_KI$	0
$PID_POS_X_KD$	0
$velMaxOverhead$	1.1
$PID_POS_VEL_X_MAX$	1 (m/s)
$\pm LIMIT$	1.1 (m/s)

Table 7 : Velocity X, PI Controller.

Parameter	Value
$PID_VEL_X_KP$	25.0
$PID_VEL_X_KI$	1.0
$PID_VEL_X_KD$	0
$rpLimitOverhead$	1.1

<i>PID_VEL_PITCH_MAX</i>	20.0 (deg)
$\pm OUTPUT LIMIT$	22 .0(deg)

Table 8 : Position Y, P Controller.

Parameter	Value
<i>PID_POS_Y_KP</i>	2.0
<i>PID_POS_Y_KI</i>	0
<i>PID_POS_Y_KD</i>	0
<i>velMaxOverhead</i>	1.1
<i>PID_POS_VEL_X_MAX</i>	1 (m/s)
$\pm LIMIT$	1.1 (m/s)

Table 9 : Velocity Y, PI Controller.

Parameter	Value
<i>PID_VEL_X_KP</i>	-25.0
<i>PID_VEL_X_KI</i>	-1.0
<i>PID_VEL_X_KD</i>	0
<i>rpLimitOverhead</i>	1.1
<i>PID_VEL_ROLL_MAX</i>	20.0 (deg)
$\pm OUTPUT LIMIT$	22 .0(deg)

Finally, it is seen that the coefficients of the velocity Y PI controller are negative. This is due to the fact that the aircraft must nose down to close the Y axis error. In other words, for positive Y movement, the aircraft must make a negative pitch angle.

4.2 Inner Loop Controller

In this section, inner loop controllers ϕ, θ, ψ attitude controllers and the structures that regulate the reference attitude rates produced by this controller will be discussed.

4.2.1 Attitude & Rate Controller

The attitude controller generates setpoints for angular velocities around the X and Y axes to regulate the rate controller, enabling the quadcopter to maintain stability at the desired angular position. The attitude rate PID controller is the one that directly controls the attitude rate. It receives almost directly the gyroscope rates (through a bit of filtering first) takes the error between the desired attitude rate as input. Then the attitude controller info given in Figure 36. The blocks contain simple PID controllers. The internal details are omitted for brevity.

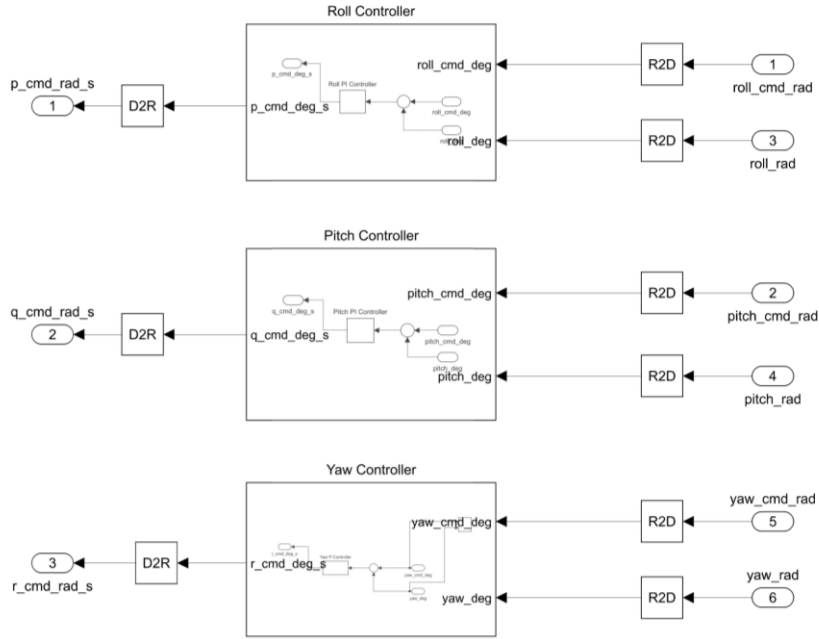


Figure 36 : Attitude controller scheme.

The integral limit for all controllers is defined in the official firmware and these values are applied to the blocks given in the simulink.

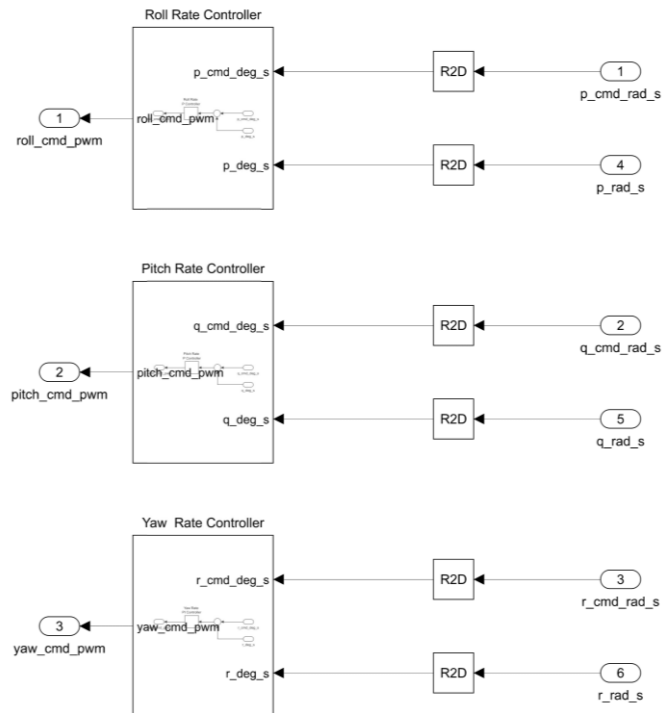


Figure 37 : Rate controller scheme.

Roll controller calculate desired roll rate reference from roll error which is PI controller. Then roll rate controller calculate desired PWM output for roll command from roll rate error signal which is PID controller. Pitch controller calculate desired pitch rate reference from roll error which is PI controller. Then pitch rate controller calculate desired PWM output for pitch command from pitch rate error signal which is PID controller. The information of roll & pitch controller given in Table 10 and Table 11.

Unlike other controllers, in the attitude controllers, a limit has been placed on the integrator output for integrator saturation. However, no limit has been applied to the outputs of the controllers in the inner loop. This is because rates can be momentarily excessive and we cannot definitively determine them. Nevertheless, the aircraft has maximum roll and pitch angles. The controller coefficients are the same for roll and pitch controllers.

Table 10 : Pitch and Roll Controller, PI Controller.

Parameter	Value
<i>PID_PITCH/ROLL_KP</i>	6.0
<i>PID_PITCH/ROLL_KI</i>	3.0
<i>PID_PITCH/ROLL_KD</i>	0
$\pm$ PID_PITCH/ROLL_INTEGRATION_LIMIT	20

Table 11 : Pitch and Roll Rate Controller, PID Controller.

Parameter	Value
<i>PID_PITCH/ROLL_RATE_KP</i>	250
<i>PID_PITCH/ROLL_RATE_KI</i>	500
<i>PID_PITCH/ROLL_RATE_KD</i>	2.5
$\pm$ PID_PITCH /ROLL_RATE_INTEGRATION_LIMIT	33.3

Yaw controller calculate desired yaw rate reference from yaw error which is PID controller. Then yaw rate controller calculate desired PWM output for yaw command from yaw rate error signal which is PI controller. Controller information is given in Table 12 and Table 13. Yaw dynamics can become more complex to control as it is governed by the total moment on the body.

Table 12 : Yaw Controller, PID Controller.

Parameter	Value
-----------	-------

PID_YAW_KP	6.0
PID_YAW_KI	1.0
PID_YAW_KD	0.35
$\pm PID_YAW_INTEGRATION_LIMIT$	360

Table 13 : Yaw Rate Controller, PI Controller.

Parameter	Value
$PID_YAW_RATE_KP$	120
$PID_YAW_RATE_KI$	16.7
$PID_YAW_RATE_KD$	0
$\pm PID_YAW_RATE_INTEGRATION_LIMIT$	166.7

These controllers, called Roll, Pitch and Yaw Rate, actually control body velocities. For this reason, it should be noted that the reference values produced are p, q, r .

The output of these controllers is 16-bit PWM command. PWM signals coming from attitude controllers are subjected to constrain processing before going to the motor mixer algorithm. The signed 16-bit constrain process for all signals is as follows:

$$(\phi, \theta, \psi)_{pwm} = \text{constrain}(\text{signal}, +32767, -32767) \quad (4.60)$$

$$(\Omega)_{pwm} = \text{constrain}(\text{signal}, 0, +65535) \quad (4.61)$$

4.3 Motor Mixer Algorithm

The first issue to be addressed is how to calculate Motor PWMs. Considering the incoming motion dynamics of the drone here, the motor 16-bit PWM inputs can be calculated via the X structure mentioned in the Section 2 as follows. This can be calculated using the thrust, yaw, pitch, roll commands and we will cover all the movements of the drone with the commands. In this way, the total torque and force components are transmitted to the motors according to the desired motion.

$$Motor_{Front,Right} = \Omega + \psi + \theta + \phi \quad (4.63)$$

$$Motor_{Front,left} = \Omega - \psi + \theta - \phi \quad (4.64)$$

$$Motor_{Back,right} = \Omega - \psi - \theta + \phi \quad (4.65)$$

$$Motor_{Back,left} = \Omega + \psi - \theta - \phi \quad (3.66)$$

Motor mixer algorithm implementation given in Figure 38.

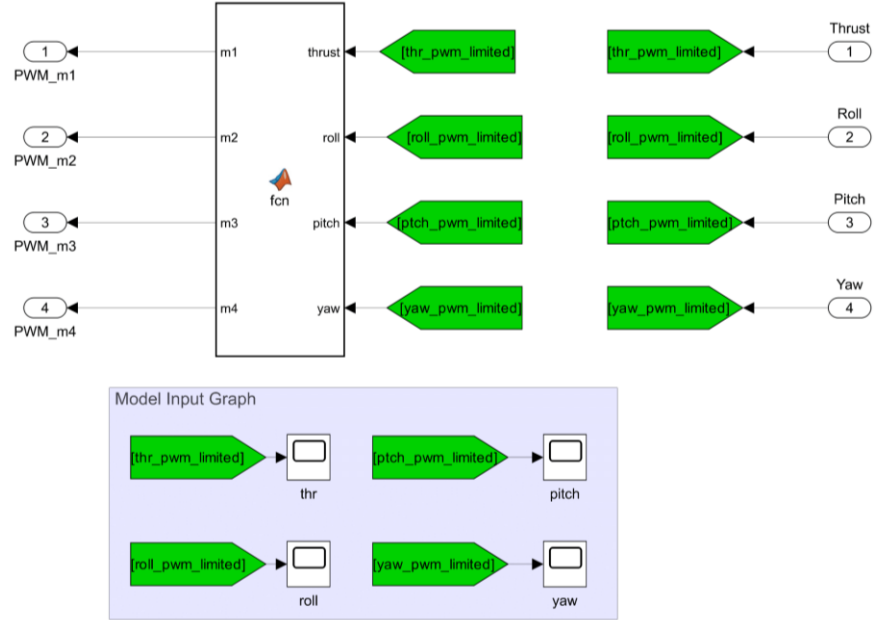


Figure 38 : Rate controller scheme.

5. SIMULATION RESULT

The established control system simulation environment constitutes the first step for aggressive trajectory generation and trajectory tracking controller designs. After the validation of this infrastructure, it will be possible to move on to the next stage. To demonstrate the accuracy of the presented theory and model, a series of tests will be applied in the simulation.

Initially, a step input will be applied to the z positions. While these inputs are applied, white noise will be introduced to observe the controller's robustness against noise. The result graphs for altitude control are provided in Figure 39.

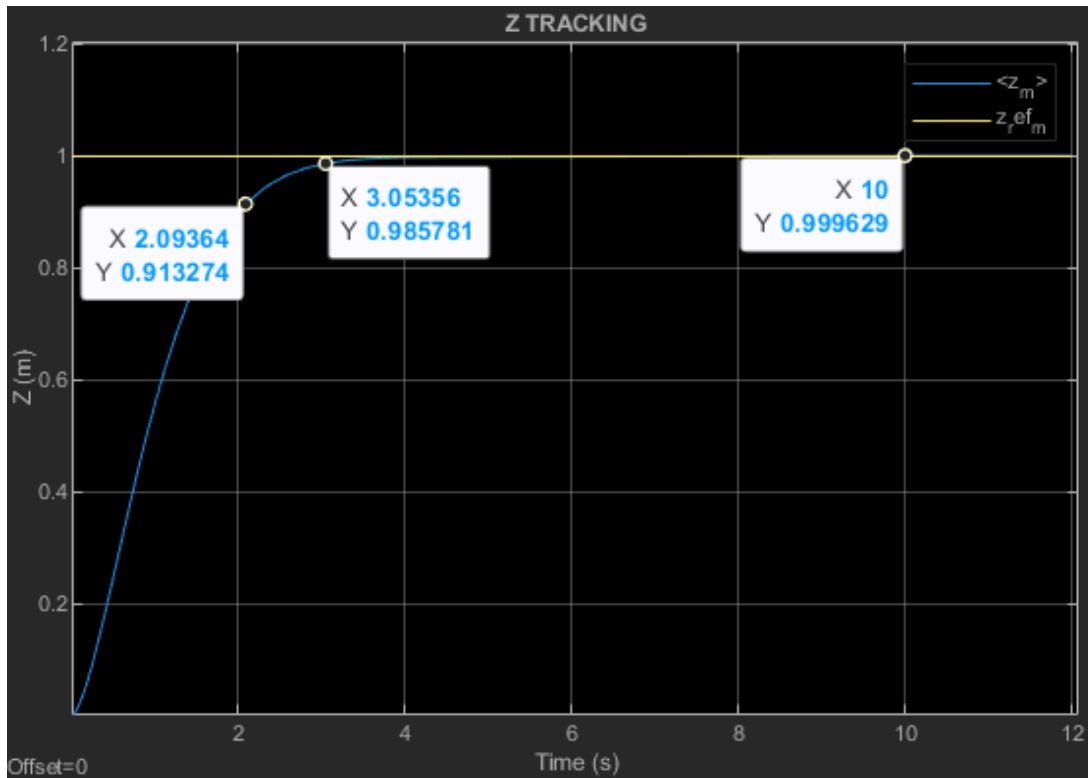


Figure 39 : Z position tracking for step input 1m.

The time-domain specifications for the system's standalone altitude tracking are as follows: The system does not exhibit overshoot, the rise time is 2.09 seconds, and the settling time is 3.0536 seconds. The steady-state error is nearly zero.

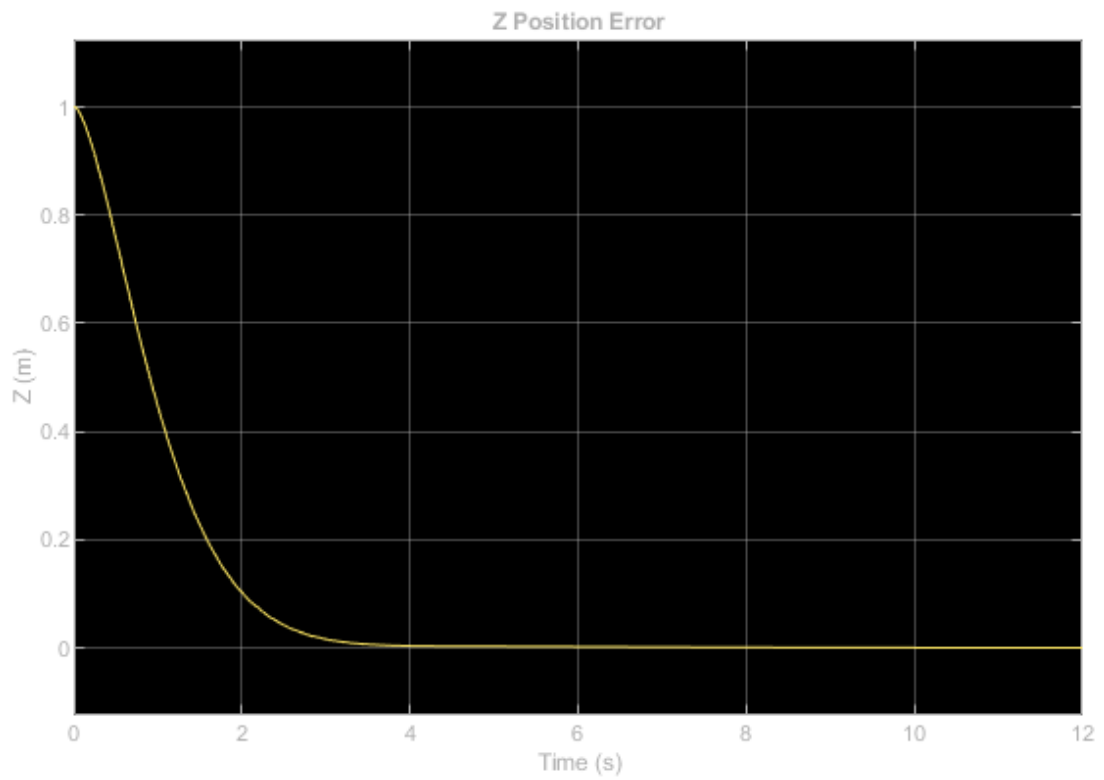


Figure 40 : Z position error for step input 1m.

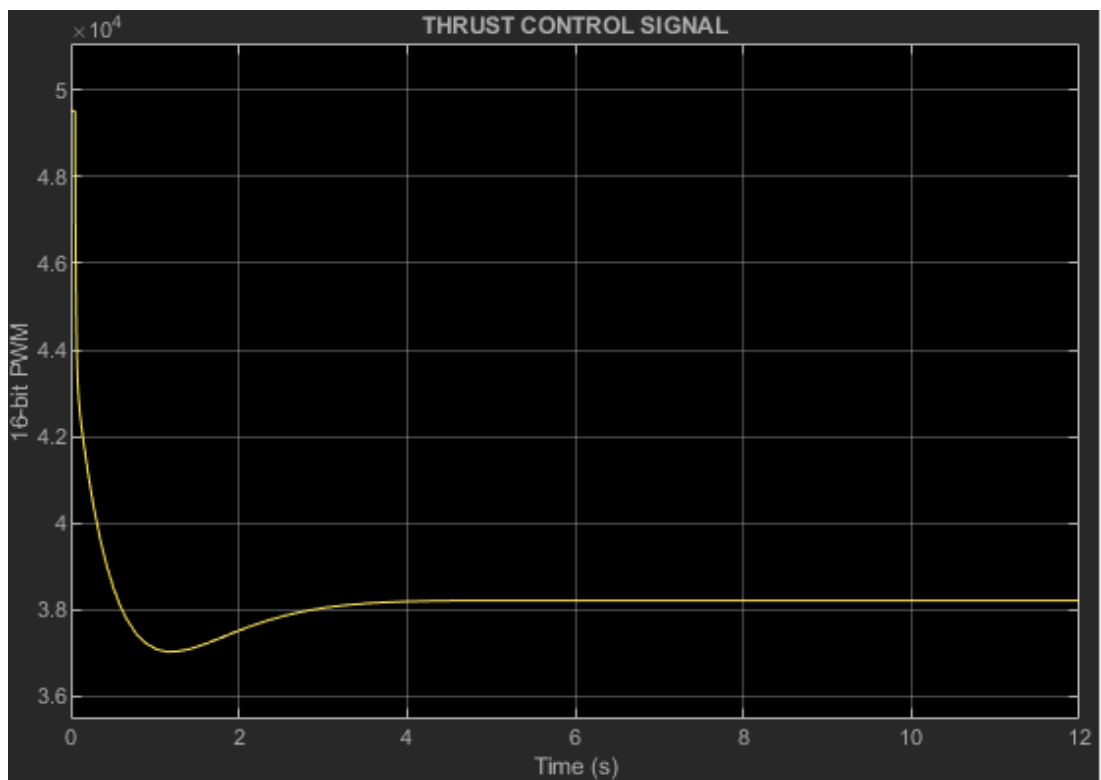


Figure 41 : For Z tracking step input 1m, control signal.

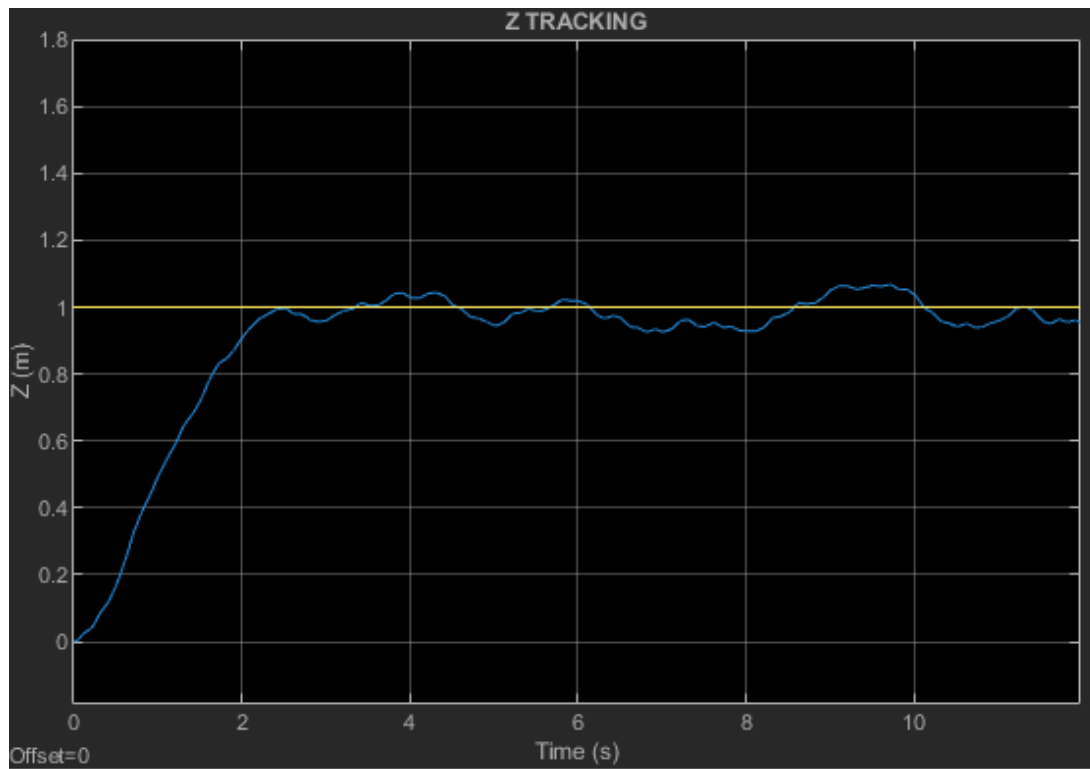


Figure 42 : Z position tracking with white noise added state estimation (0.001 power) step input 1m.

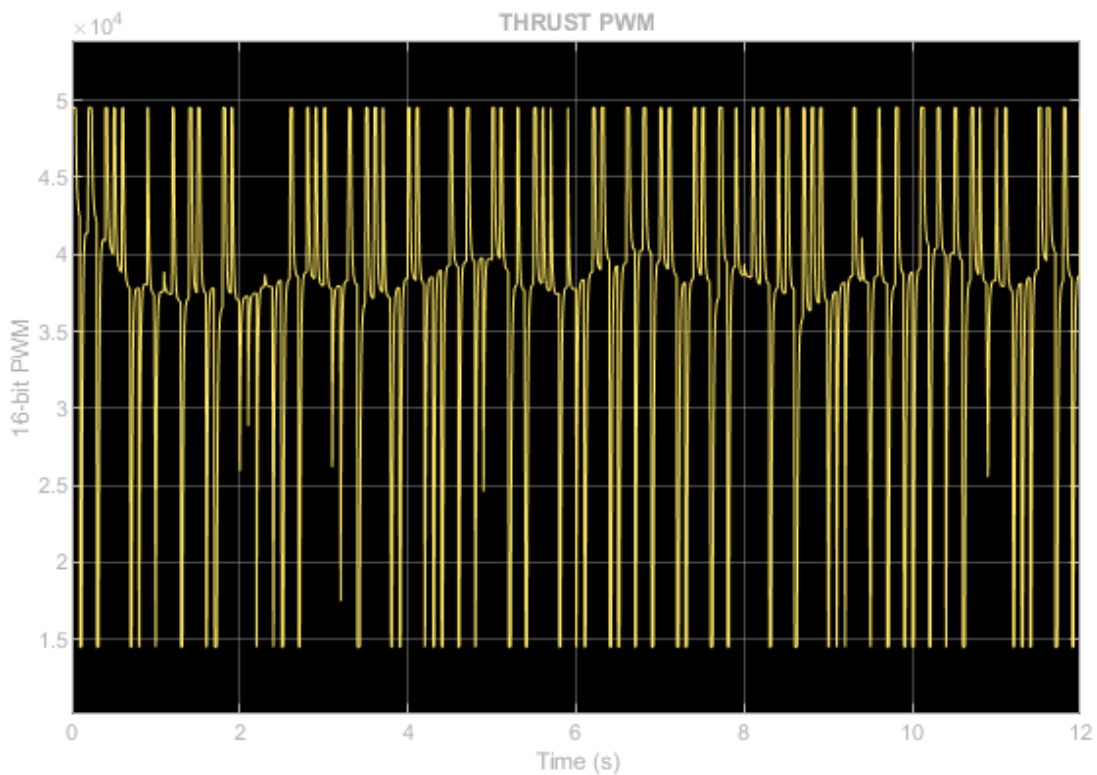


Figure 43 : For Z tracking with white noise added state estimation (0.001 power) step input 1m, control signal.

Another test case involves applying a step input at various times for all axes. The reference signals are shown in Figure 44.

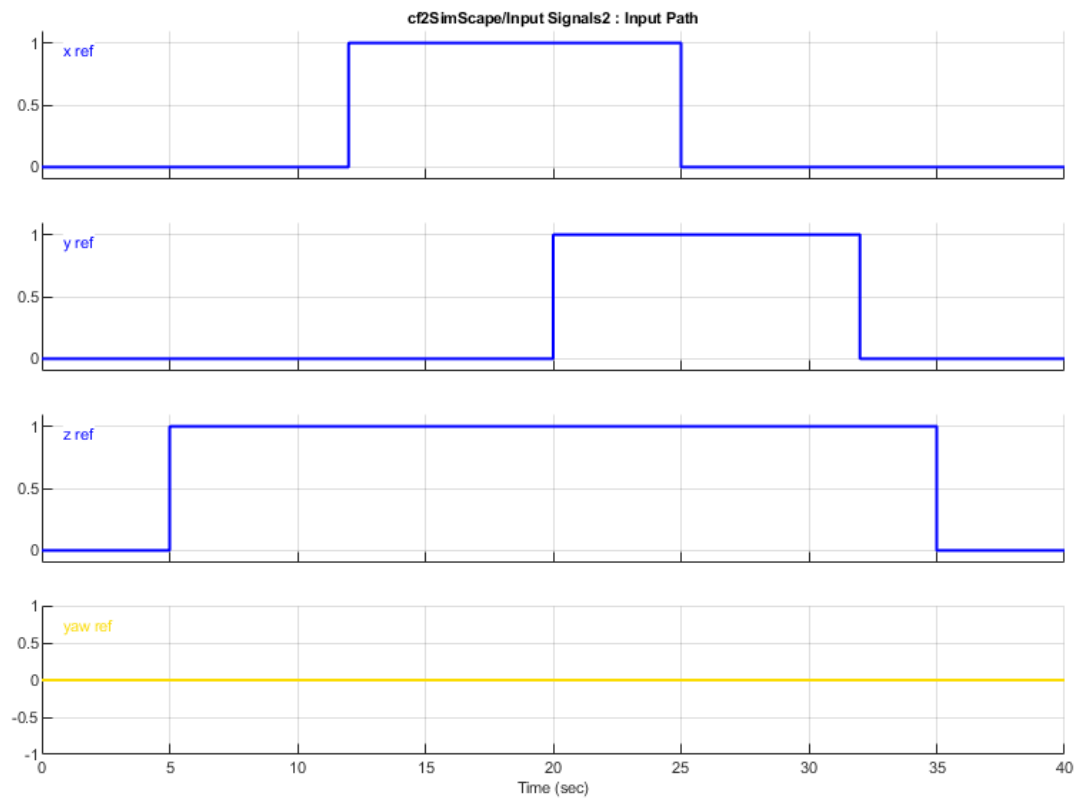


Figure 44 : Reference Signal for position step tracking.

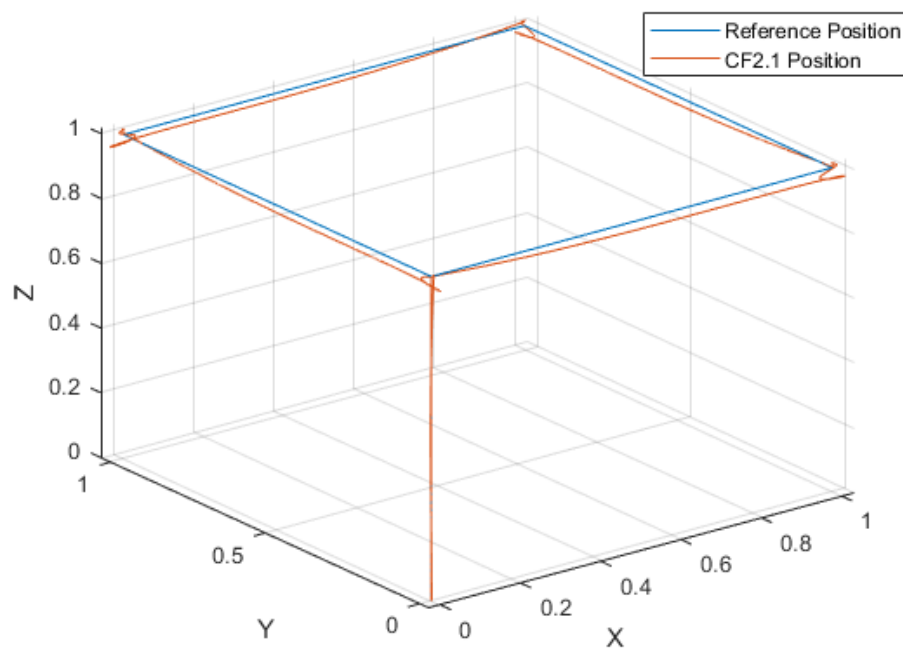


Figure 45 : X,Y,Z step tracking simulation.

The yaw reference has been kept constant throughout all tests. It was kept constant due to an issue with the yaw control in the model. The yaw controllers operate with a reference of 0. The outputs for the reference signals provided in Figure 44 are as follows.

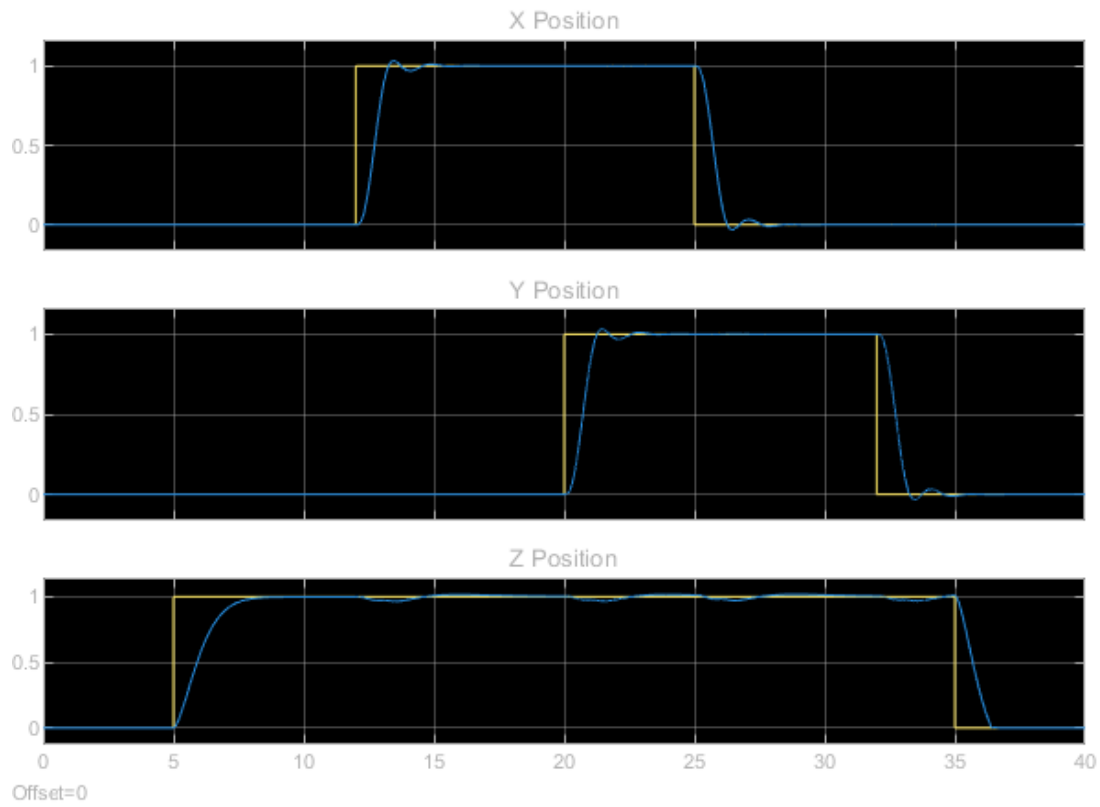


Figure 46 : X,Y,Z step tracking performance.

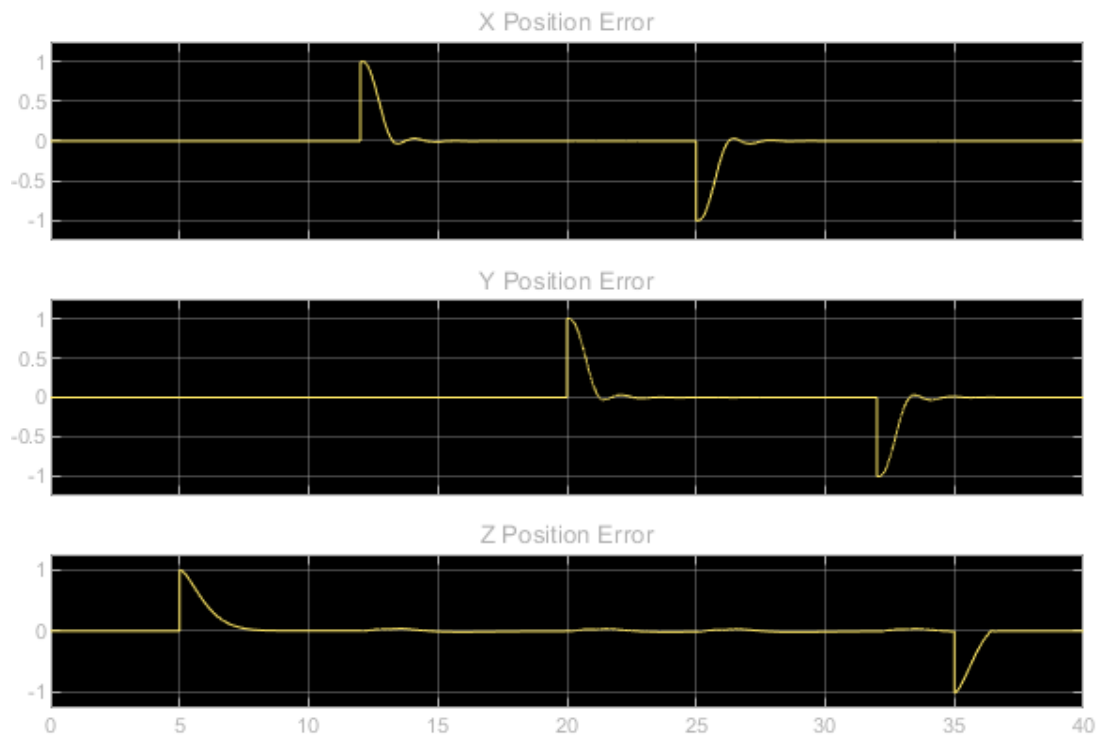


Figure 47 : X,Y,Z step tracking error.

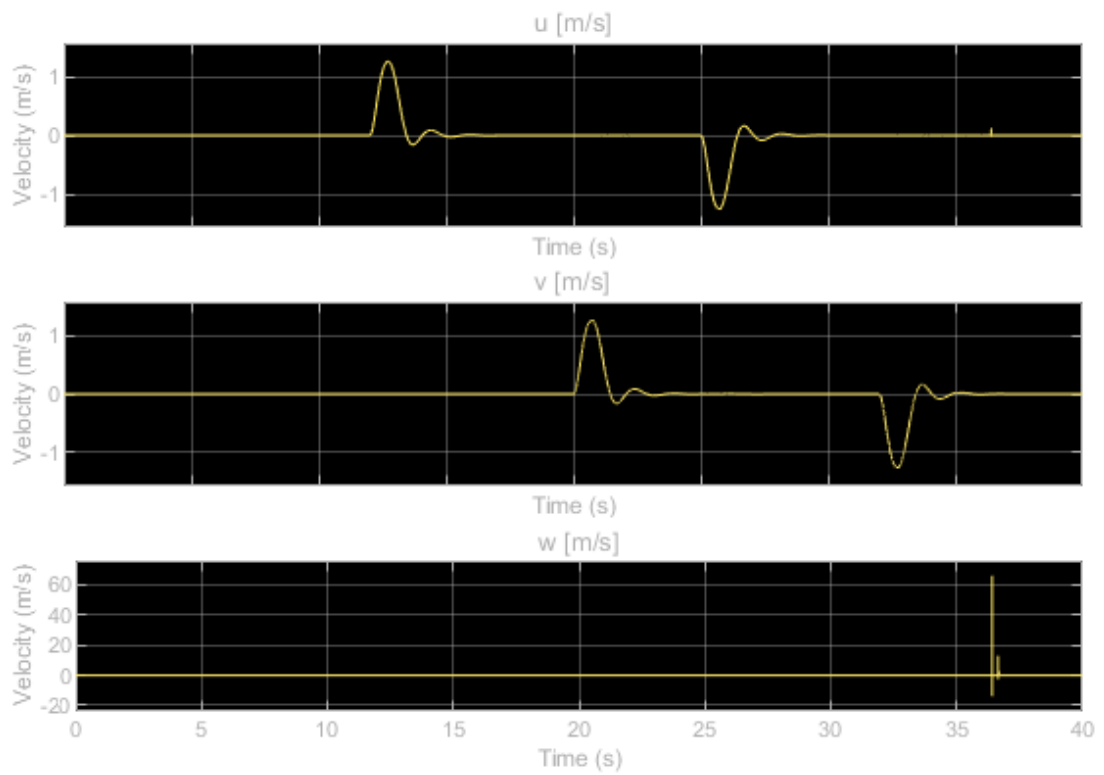


Figure 48 : X,Y,Z step tracking result body velocity.

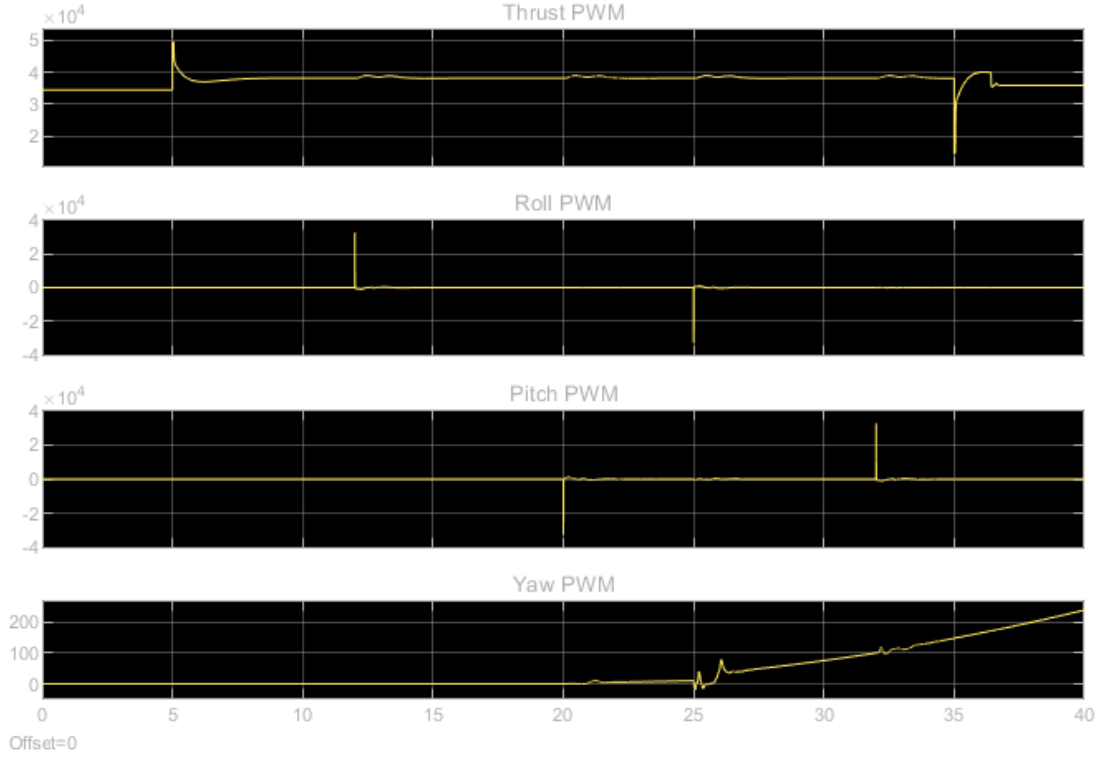


Figure 49 : X,Y,Z step tracking control signals.

As seen in Figure 49, there is a clear issue with the yaw control. The source of the problem has not yet been identified. Considering that the primary factor influencing position is velocity, speed graphs play an important role in trajectory tracking for this reason.

In order to effectively demonstrate the necessity of utilizing a more advanced trajectory tracker, the simulated system was tasked with following a more intricate trajectory to assess its tracking performance. Bu amaçla helix trajectory testi tanımlanmıştır.

$$x = r \cos(\omega t), r = 0.5m, \omega = \left(\frac{2\pi}{5}\right) \quad (4.67)$$

$$y = r \sin(\omega t), r = 0.5m, \omega = \left(\frac{2\pi}{5}\right) rad/sec \quad (4.68)$$

$$z = ct, c = 0.3m/sec \quad (4.69)$$

Where, r is the fixed radius of the helix, ω the angular frequency, corresponding to one full rotation and c correspond to altitude rate in 1 second. The results of the test are provided in the following graphs.

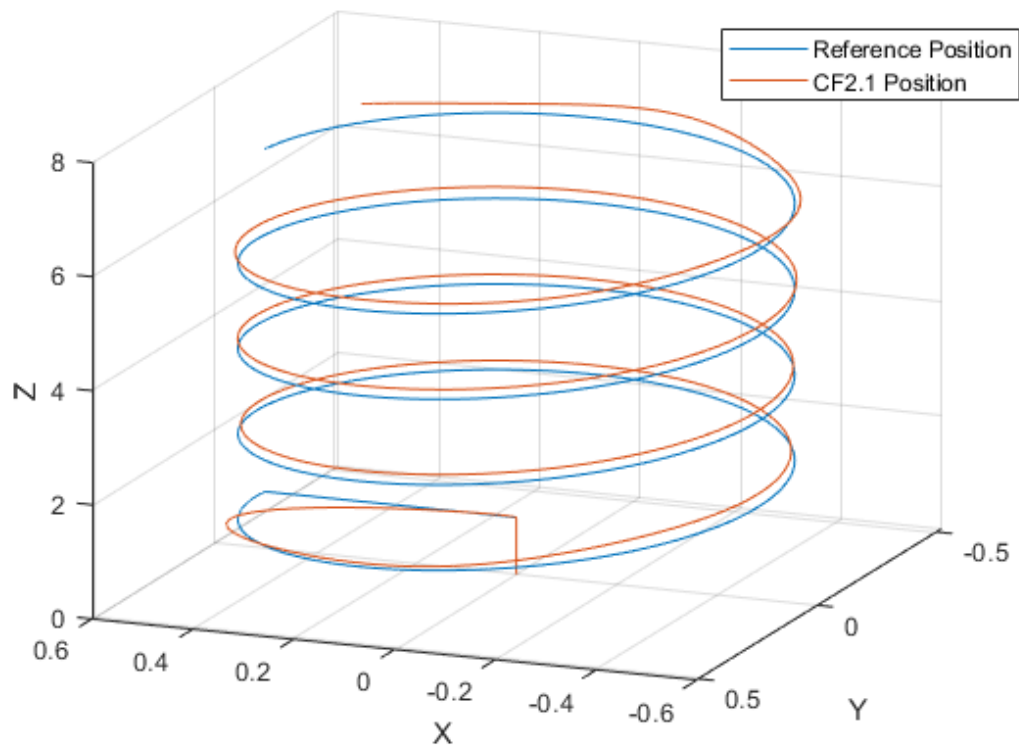


Figure 50 : Helix trajectory tracking simulation.

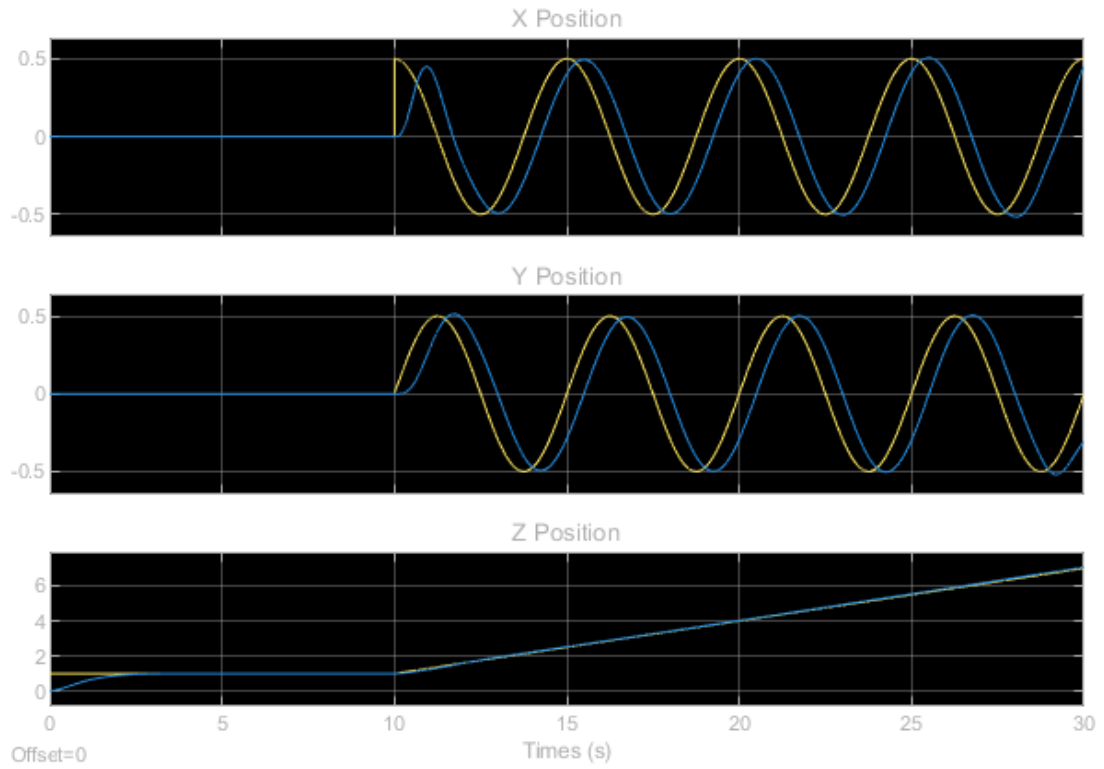


Figure 51 : Helix trajectory tracking performance.

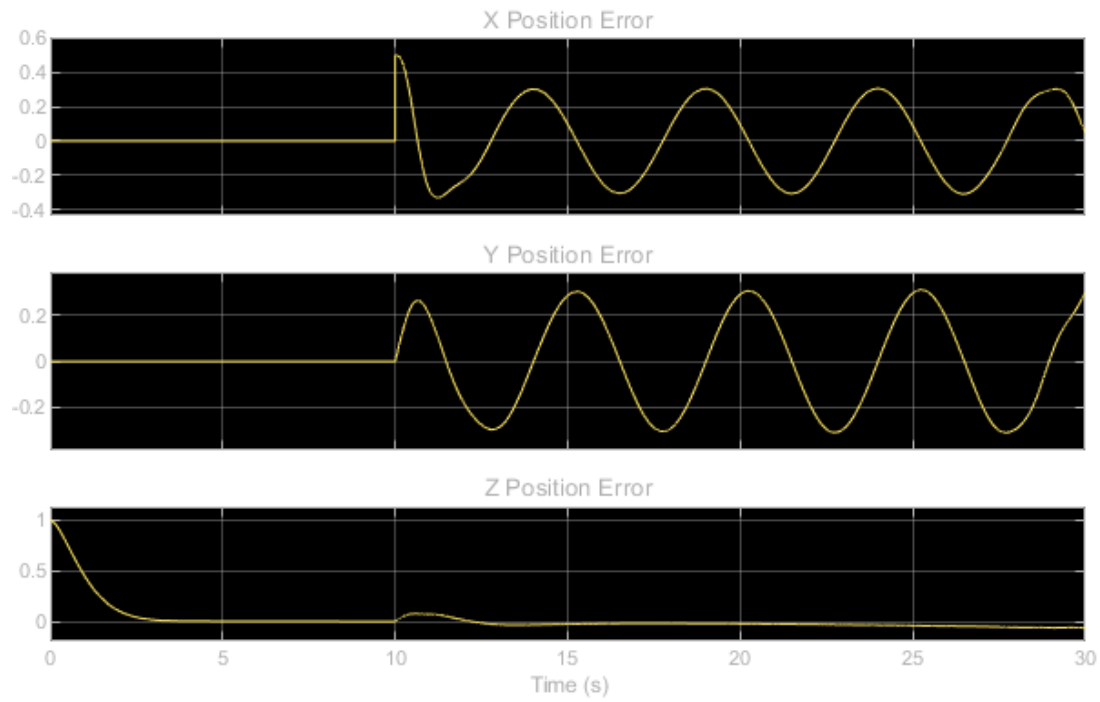


Figure 52 : Helix trajectory tracking error.

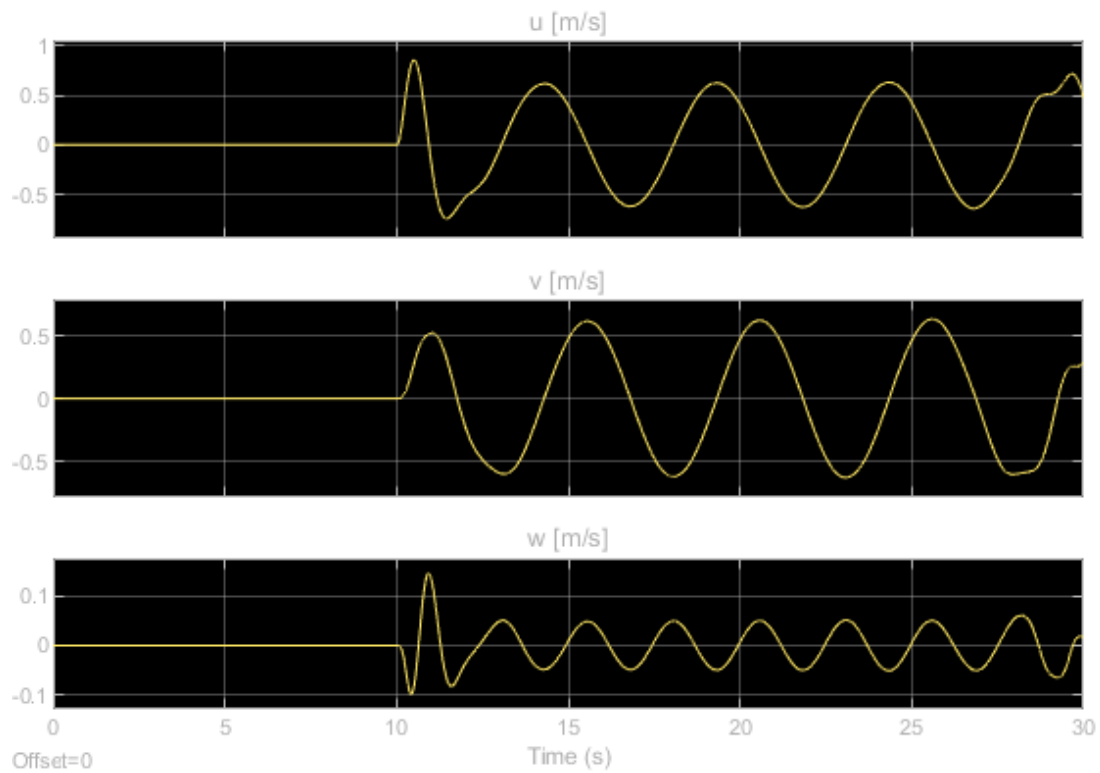


Figure 53 : Helix trajectory tracking result body velocity.

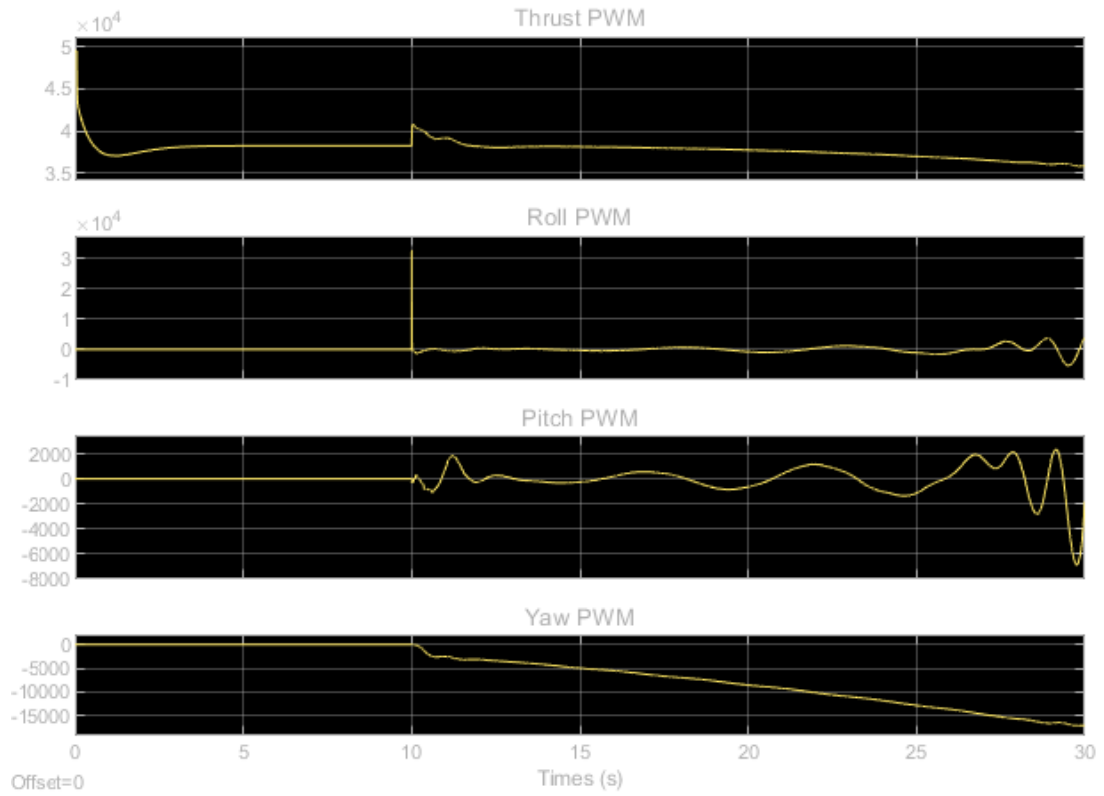


Figure 54 : Helix trajectory tracking control signals.

The results clearly show that in the tracking of the X and Y axes, the tracking performance remains slow for the given angular velocity, and a phase difference occurs. In the Z position, although there is a persistent but small position error, the majority of the error can be eliminated.

6. CONCLUSION

The focus of this study was to develop aggressive motion planning algorithms and implement them on a platform by following the most recent mathematical advancements in the literature. However, the study shifted its primary focus toward building a simulation model and understanding the mathematical modeling of aerial vehicles. As a result, the study evolved into a detailed mathematical modeling effort and the development of a common controller structure and its cycles. Ultimately, an output was achieved that is known to function and provides an environment for testing. This shift in focus increased the distance from the original goal, and the desired progress in those areas could not be achieved. However, as of this point, I have reached a stage where I can design and implement new controllers and understand the thought process behind designing these controllers internally.

For aggressive trajectory planning and implementation, two fundamental requirements have emerged. The first is to design a controller capable of following a trajectory generated for aggressive path tracking. While there is no fixed or well-established structure for this, research is ongoing. The main focus again goes back to the dynamic equations of the aerial vehicle. Dynamics theory tells us that the primary cause of a rigid body's (which itself is an approximation) position is its velocity, and the primary cause of velocity is acceleration. Without acceleration, no velocity profile can be formed, and no desired position can be achieved. Trajectory generation algorithms aim to answer the question of how an aerial vehicle moves from one point to another by tracing this process backward. The focus lies on questions such as: "What acceleration profile should I use to achieve a desired position sequence or an aggressive maneuver?" and "How will I generate this acceleration profile?".

One of the basic techniques that emerges while generating this acceleration profile is minimizing snap, which is the derivative of acceleration. Additional questions also arise, such as whether this acceleration profile should be planned specifically according to the dynamics of the aerial vehicle or should it follow a general method. Ultimately, since the vehicle will follow this profile, the algorithm must take the dynamics of the aerial vehicle into account, which falls under the topic of feasibility assurance. After generating the acceleration profile, the next question is how the given

vehicle will follow this acceleration. The controller presented in this study does not produce or consider any form of acceleration generation. Instead, position tracking is attempted through rate commands, which is a relatively simple method. While this method has been shown to perform well in basic position tracking, it is not considered a state-of-the-art method. Consequently, developing a new controller that focuses on acceleration tracking appears to be a good option, although it cannot be definitively stated as the correct path. However, upon examining the literature, it is evident that there is a growing focus on acceleration tracking.

For a quadcopter or fixed-wing aerial vehicle, acceleration tracking emerges as a situation that requires an in-depth consideration of the vehicle's entire dynamics, including both translational and rotational dynamics, as well as their interactions (e.g., angular velocity controllers, quaternion controllers, etc.). By conducting further studies to meet these two requirements, new methods and techniques can be considered, contributing to the evolving mathematics of aggressive trajectory planning.

Ultimately, regardless of the platform being used, a solid theoretical foundation is essential. Along this journey, this project has served as a guide for understanding aerial vehicle control and its underlying mathematics. As a result, a simulation environment platform was successfully developed.

REFERENCES

- [1] **Preiss, J. A., Hönig, W., Sukhatme, G. S., & Ayanian, N.** (2017, May). CrazySwarm: A large nano-quadcopter swarm. In 2017 IEEE International Conference on Robotics and Automation (ICRA) (pp. 3299-3304). IEEE.
- [2] **Autonomous Microrobotic Systems Laboratory USC.** (2016, March 20). Multi-flipping maneuvers about two different axes by using Crazyflie 1.0 [Video]. YouTube.
<https://www.youtube.com/watch?v=MzCNoX4yTZg>.
- [3] **Förster, J.** (2015). System identification of the crazyflie 2.0 nano quadcopter (Bachelor's thesis, ETH Zurich).
- [4] **Luis, C., & Ny, J. L.** (2016). Design of a trajectory tracking controller for a nanoquadcopter. arXiv preprint arXiv:1608.05786.
- [5] **Geisert, M., & Mansard, N.** (2016, May). Trajectory generation for quadrotor based systems using numerical optimal control. In 2016 IEEE international conference on robotics and automation (ICRA) (pp. 2958-2964). IEEE.
- [6] **Jain, S.** (2023). Towards Model Predictive Control for Acrobatic Quadrotor Flights (Master's thesis, Arizona State University).
- [7] **Lambert, N. O., Drew, D. S., Yaconelli, J., Levine, S., Calandra, R., & Pister, K. S.** (2019). Low-level control of a quadrotor with deep model-based reinforcement learning. *IEEE Robotics and Automation Letters*, 4(4), 4224-4230.
- [8] **Mellinger, D., & Kumar, V.** (2011, May). Minimum snap trajectory generation and control for quadrotors. In 2011 IEEE international conference on robotics and automation (pp. 2520-2525). IEEE.
- [9] **Ryou, G., Tal, E., & Karaman, S.** (2023, March). Real-time generation of time-optimal quadrotor trajectories with semi-supervised seq2seq learning. In *Conference on Robot Learning* (pp. 1860-1870). PMLR.
- [10] **Tomić, T., Maier, M., & Haddadin, S.** (2014, May). Learning quadrotor maneuvers from optimal control and generalizing in real-time. In 2014 IEEE International Conference on Robotics and Automation (ICRA) (pp. 1747-1754). IEEE.
- [11] **Chen, Y., & Pérez-Arancibia, N. O.** (2016, May). Generation and real-time implementation of high-speed controlled maneuvers using an autonomous 19-gram quadrotor. In 2016 IEEE international conference on robotics and automation (ICRA) (pp. 3204-3211). IEEE.
- [12] **Beard, R. W., & McLain, T. W.** (2012). *Small unmanned aircraft: Theory and practice*. Princeton university press.

- [13] **Mahony, R., Kumar, V., & Corke, P.** (2012). Multicopter aerial vehicles: Modeling, estimation, and control of quadrotor. *IEEE robotics & automation magazine*, 19(3), 20-32.
- [14] **Bry, A., Richter, C., Bachrach, A., & Roy, N.** (2015). Aggressive flight of fixed-wing and quadrotor aircraft in dense indoor environments. *The International Journal of Robotics Research*, 34(7), 969-1002.
- [15] **Carlos, B. B., Sartor, T., Zanelli, A., Frison, G., Burgard, W., Diehl, M., & Oriolo, G.** (2020, December). An efficient real-time NMPC for quadrotor position control under communication time-delay. In *2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV)* (pp. 982-989). IEEE.
- [16] **Lupashin, S., Schöllig, A., Sherback, M., & D'Andrea, R.** (2010, May). A simple learning strategy for high-speed quadcopter multi-flips. In *2010 IEEE international conference on robotics and automation* (pp. 1642-1648). IEEE.
- [16] **Silano, G., & Iannelli, L.** (2020). CrazyS: A software-in-the-loop simulation platform for the crazyflie 2.0 nano-Quadcopter. *Robot Operating System (ROS) The Complete Reference (Volume 4)*, 81-115.